An Internship Report On

"Personal Carbon Footprinting Application"

BY

Mr. Akshay Shankar Tarpe

Under the Guidance

of

Ms. P. S. Bihade

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Mahatma Gandhi Mission's College of Engineering, Nanded (M.S.)

Academic Year 2025-26

An Internship Report on

"Personal Carbon Footprinting Application"

Submitted to

DR. BABASAHEB AMBEDKAR TECHNOLOGICAL UNIVERSITY, LONERE

in fulfillment of the requirement for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

By

Mr. Akshay Shankar Tarpe

Under the Guidance
of

Ms. P. S. Bihade

(Department of Computer Science and Engineering)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MAHATMA GANDHI MISSION'S COLLEGE OF ENGINEERING
NANDED (M.S.)

Academic Year 2025-26

Certificate

This is to certify that the internship entitled

"Personal Carbon Footprinting Application"

being submitted by ***Mr. Akshay Shankar Tarpe*** *to the Dr. Babasaheb Ambedkar Technological University, Lonere , for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by him under my supervision and guidance. The matter contained in this report has not been submitted to any other university or institute for the award of any degree.*

Ms. P. S. Bihade

Project Guide

Dr. A. M. Rajurkar

H.O.D

Computer Science & Engineering

Dr. G. S. Lathkar

Director

MGM's College of Engg., Nanded

INFOSYS VIRTUAL INTERNSHIP 6.0 OFFER LETTER

Welcome to the Infosys Internship Program | Project: Personal Carbon Footprint App

1 message

Ajit <springboardmentor8888@gmail.com> Tue, Feb 3, 2026 at 7:48 PM
Bcc: s22_tarpe_akshay@mgmcen.ac.in

Dear Team,

Congratulations on your selection, and welcome to the Infosys Internship Program!

My name is Ajit Kumar, and as a Software Development Engineer, I am pleased to be your mentor for this engagement. I look forward to working with you on our assigned project: the **"Personal Carbon Footprint App."**

To initiate this project, our first virtual meeting is scheduled as follows:

Date: Thursday, 5th February 2026
Time: 6:45 PM – 7:30 PM

Operational Guidelines & Expectations:

Daily Cadence: Commencing 5th February 2026, we will hold daily sync-ups Monday through Friday during this same time slot (6:45 PM - 7:30 PM).

Meeting Format: Please note that these sessions will not follow a lecture format. They are designed as collaborative discussions focused on guidance, removing blockers, and problem-solving. Active participation is expected.

Initial Agenda: Our first few sessions will be dedicated to team introductions, project architecture analysis, setting milestones, and addressing any immediate skill gaps.

Attendance Policy: Professional punctuality is required. Should you need to take leave, please inform me in advance.

Communication: All official correspondence regarding the project should be conducted via email to ensure proper documentation.

I will send a separate calendar invitation with the meeting link shortly. I look forward to meeting you all and kicking off a successful project.

Best Regards,
Ajit Kumar
Mentor – Infosys Internship Program

INFOSYS VIRTUAL INTERNSHIP 6.0 COMPLETION LETTER


Infosys
Navigate your next

CERTIFICATE OF COMPLETION

presented to

Akshay Tarpe

for actively participating and completing the mandatory assignment related to

Internship 6.0 (B 13): Individual Carbon Footprint Monitoring and Reduction Guidance Application

conducted from February 5, 2026 to April 3, 2026

Infosys | Springboard

Issued on: Friday, May 29, 2026
To verify, scan the QR code at https://verify.onwingspan.com

Satheesha B.N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my project guide, **Ms. P. S. Bihade**, for her invaluable support, guidance, and encouragement throughout this project. Her profound knowledge and expertise have been instrumental in the successful completion of this work. Her patience and willingness to assist me at every step have greatly enriched my learning experience. Her constructive feedback and insightful suggestions have not only helped me overcome challenges but also motivated me to strive for excellence.

I would like to express my sincere gratitude to Infosys Springboard for providing me with the opportunity to undertake the Infosys Springboard Virtual Internship 6.0. I am especially thankful to my mentor, **Mr. Ajit Kumar**, for his guidance, support, and valuable insights throughout the internship. The knowledge and practical experience gained during this period significantly contributed to the successful development of my project, Personal Carbon Footprinting Application, and enhanced my technical and professional skills.

I gladly take this opportunity to thank **Dr. A. M. Rajurkar** (Head of Computer Science & Engineering, MGM's College of Engineering, Nanded). I am heartily thankful to **Dr. G. S. Lathkar** (Director, MGM's College of Engineering, Nanded) for providing facilities during the progress of the project and also for her kind help, guidance and inspiration. Last but not least, I am also thankful to all those who helped, directly or indirectly, develop this project and complete it successfully.

With Deep Reverence,

Mr. Akshay Shankar Tarpe

[B.Tech CSE-A]

ABSTRACT

Environmental pollution and climate change have become major global concerns due to increasing carbon emissions generated through transportation, electricity consumption, industrial growth, and unsustainable lifestyle activities. Many individuals are unaware of how their daily habits contribute to environmental degradation because they lack proper systems for monitoring and analyzing their personal carbon footprint. Existing carbon tracking systems often provide only basic calculations without interactive analytics, long-term monitoring, or user engagement features. The Personal Carbon Footprint Application was developed to address these limitations by providing an interactive and user-friendly sustainability monitoring platform. The application allows users to track carbon emissions generated through transportation usage, electricity consumption, and lifestyle activities. Based on user inputs, the system calculates estimated carbon emissions and displays the results using graphical dashboards, charts, and analytical reports.

The application also includes advanced engagement features such as sustainability goals, achievement badges, progress tracking, and leaderboard systems to motivate users toward eco-friendly behavior. JWT authentication and secure API communication were implemented to protect user information and improve application security. The system was developed using modern full-stack technologies including React.js for frontend development, Spring Boot for backend API implementation, and PostgreSQL for database management. REST APIs were used to establish communication between frontend and backend modules.

The project demonstrates how modern web technologies can be utilized to create socially meaningful applications that promote environmental awareness and sustainable living practices. By combining sustainability tracking with interactive user engagement features, the application encourages users to adopt environmentally responsible habits and contribute positively toward environmental protection.

TABLE OF CONTENTS

LIST OF FIGURES

Chapter 1

INTRODUCTION

Environmental pollution and climate change have become serious global issues in recent years. Rapid industrial growth, increasing vehicle usage, urbanization, and excessive energy consumption are continuously affecting the environment. Human activities release large amounts of greenhouse gases into the atmosphere, which increases global warming and creates environmental imbalance. These environmental changes are affecting weather conditions, natural resources, agriculture, and public health across many countries. Carbon dioxide is one of the major greenhouse gases responsible for climate change [10]. Every individual contributes to carbon emissions directly or indirectly through daily activities such as transportation, electricity usage, food consumption, and fuel burning. Although people use these resources regularly, many do not understand how their lifestyle contributes to environmental pollution because they lack proper monitoring systems.

The Personal Carbon Footprint Application was developed to address this problem by helping users track and analyze their carbon emissions. The application allows users to enter details related to transportation, electricity consumption, and lifestyle habits. Based on this information, the system estimates carbon emissions and displays the results using dashboards, charts, and graphical analytics. Unlike traditional awareness systems that only provide educational information, this application focuses on practical monitoring and interactive user engagement. Users can regularly monitor their progress, set reduction goals, and compare performance using leaderboard systems. The project combines environmental sustainability concepts with modern web technologies to create a useful and engaging digital platform.

The application was developed using React.js for frontend development, Spring Boot for backend API implementation, and PostgreSQL for database management. JWT authentication was also implemented to ensure secure user login and protected API communication. The project demonstrates how software systems can contribute toward solving environmental challenges by encouraging users to adopt sustainable lifestyle practices.

1.1 Background

Climate change has become one of the most discussed environmental concerns globally. Rising temperatures, melting glaciers, irregular rainfall, air pollution, and extreme weather conditions are some of the visible effects of environmental imbalance. One major reason behind these issues is the increasing amount of greenhouse gas emissions generated through human activities. Transportation systems, electricity production, industrial operations, and fuel consumption are among the largest contributors to carbon emissions. At the same time, common lifestyle activities such as excessive use of electrical appliances, private vehicle usage, and unnecessary energy consumption also contribute significantly to environmental pollution.

Many organizations and governments are promoting sustainable development and eco-friendly practices. However, awareness alone is often not enough because users need practical tools to understand their environmental impact. Most people cannot estimate how much carbon they generate daily through routine activities. Several carbon footprint calculators are already available online, but many existing systems provide only basic calculations without long-term tracking or user engagement features. They often lack visual analytics, goal tracking, and motivational elements. Due to this limitation, users lose interest quickly and fail to monitor their environmental behavior consistently.

The Personal Carbon Footprint Application was created to improve this situation by offering an interactive platform that combines carbon tracking with user engagement features such as dashboards, achievements, and leaderboards. The system aims to transform sustainability awareness into regular user activity through practical monitoring and visualization. The project also reflects the growing role of technology in addressing social and environmental problems. Modern software applications can influence user behavior positively when designed effectively. By providing measurable insights and graphical analytics, the system encourages users to make environmentally responsible decisions.

1.2 Problem Statement

Environmental pollution and climate change have become major global concerns due to the continuous increase in greenhouse gas emissions generated through human activities. Rapid urbanization, industrial growth, increasing transportation usage, excessive electricity consumption, and unsustainable lifestyle practices contribute

significantly toward rising carbon emissions. These environmental issues negatively affect air quality, natural ecosystems, public health, and overall ecological balance. Among various greenhouse gases, carbon dioxide (CO_2) is considered one of the primary contributors to global warming and climate change.

Although environmental awareness has increased in recent years, many individuals still lack proper systems to monitor and analyze their personal contribution toward environmental pollution. Most people are unaware of how daily activities such as vehicle usage, electricity consumption, food habits, and energy utilization impact the environment. As a result, individuals are unable to evaluate their sustainability performance or adopt effective carbon reduction strategies.

Existing carbon footprint tracking systems available online suffer from several limitations. Many applications provide only basic carbon calculations without long-term monitoring and analytical support. Some systems contain complex interfaces that are difficult for common users to understand, while others lack interactive dashboards, progress tracking mechanisms, graphical visualizations, and user engagement features. The absence of motivational functionalities such as goals, achievements, and leaderboard systems reduces user interest and discourages continuous sustainability monitoring.

In addition, several existing applications are not optimized for responsive usage across different devices, which further affects accessibility and user experience. Another major issue is that environmental information is often presented in technical or numerical formats that are difficult for non-technical users to interpret. Complex statistical outputs reduce user engagement and make sustainability monitoring less effective. Therefore, there is a need for a simple, user-friendly, and visually interactive platform that can help users understand their environmental impact more efficiently.

To address these challenges, the Personal Carbon Footprint Application was developed as an interactive sustainability monitoring platform. The system provides functionalities such as carbon emission tracking, dashboard analytics, sustainability goal management, achievement badges, leaderboard comparisons, progress monitoring, responsive user interfaces, and secure authentication mechanisms. The application aims to improve environmental awareness by helping users monitor their daily carbon emissions and encouraging them to adopt eco-friendly habits through interactive and engaging digital features.

1.3 Objectives

The main objective of the Personal Carbon Footprint Application is to create environmental awareness by helping users monitor and analyze their carbon emissions through digital analytics and interactive sustainability tracking systems. The project aims to encourage eco-friendly habits and promote responsible environmental behavior by providing users with an easy-to-use platform for monitoring their daily activities and carbon footprint.

The project has several important objectives which are explained below:

a. To Develop a Carbon Tracking Platform

One of the primary objectives of the project is to develop a digital platform that allows users to calculate and monitor carbon emissions generated through daily lifestyle activities. The system enables users to enter information related to transportation methods, electricity consumption, and food habits, which are then processed to estimate carbon footprint values. The platform helps users understand how everyday activities contribute toward environmental pollution and global warming. By maintaining carbon history records and dashboard summaries, the system supports continuous environmental monitoring instead of one-time calculations.

b. To Increase Environmental Awareness

Another important objective of the project is to improve environmental awareness among users regarding sustainability and climate change. Many individuals are unaware of how their routine activities impact the environment. The application educates users by presenting carbon emission data in a simplified and understandable manner. Through regular monitoring and visual analytics, users become more conscious about reducing energy consumption, minimizing pollution, and adopting sustainable lifestyle practices. The project encourages individuals to participate actively in environmental protection through digital awareness systems.

c. To Provide Visual Analytics and Dashboard Reporting

The project aims to simplify environmental monitoring through graphical visualizations and dashboard analytics instead of displaying only numerical values. Charts, emission summaries, progress indicators, and analytical reports help users understand sustainability data more effectively. Visual dashboards improve readability and make complex environmental information easier to interpret for users from different technical

backgrounds. The analytical interface also improves user interaction and supports better decision-making regarding carbon reduction strategies.

d. To Encourage Eco-Friendly Habits Through Gamification

The application was designed to motivate users toward environmentally responsible behavior by integrating gamification features such as goals, badges, achievements, progress tracking, and leaderboard systems. Users can create sustainability goals and monitor their progress regularly through interactive dashboards. Achievement badges and ranking systems encourage continuous participation and improve user engagement. These motivational features transform sustainability tracking into an interactive and rewarding experience instead of a passive monitoring process.

e. To Create a User-Friendly and Responsive Interface

Another objective of the project is to develop a simple, responsive, and user-friendly interface that can be easily used by individuals with different technical skill levels. The frontend system was designed using modern UI principles and responsive layouts to ensure smooth accessibility across desktops, tablets, and mobile devices. Proper navigation systems, organized dashboards, and form validation mechanisms improve usability and simplify user interaction throughout the application.

f. To Implement Secure Authentication and Data Protection

Security was considered an important requirement during system development. The application implements JWT (JSON Web Token) authentication to provide secure login handling and protected API communication. Password encryption mechanisms were also implemented to improve user data protection and prevent unauthorized access. Secure authentication ensures that personal sustainability records and account information remain protected throughout system usage. Proper session handling and validation mechanisms further improve application reliability and security.

g. To Demonstrate Full-Stack Web Development Implementation

The project also aims to demonstrate practical implementation of modern full-stack web development technologies and software engineering concepts. React.js was used for frontend interface development, Spring Boot for backend API implementation, and PostgreSQL for relational database management. REST API communication, authentication handling, database integration, and responsive UI development were successfully implemented during the project lifecycle. The application serves as a practical example of integrating frontend, backend, and database systems within a real-world sustainability-based software solution.

h. To Build a Scalable Sustainability Monitoring System

Another objective of the project is to design the application using scalable architecture principles so that future enhancements and advanced sustainability features can be integrated easily. The modular system structure supports future expansion such as mobile application support, AI-based recommendations, IoT-based environmental monitoring, advanced analytics, and organization-level sustainability management systems. This scalability ensures that the platform can evolve into a more comprehensive environmental management solution in the future.

1.4 Scope of the Project

The scope of the Personal Carbon Footprint Application mainly focuses on helping individual users monitor and manage their daily carbon emissions through a digital sustainability tracking platform. The system was developed to track activities that contribute significantly to environmental pollution, including transportation usage, electricity consumption, and lifestyle habits. By analyzing this information, the application helps users understand how their routine activities affect the environment and encourages them to adopt more sustainable practices.

The application provides several important functionalities designed to improve environmental awareness and user engagement. Users can register and manage personal accounts securely using authentication features integrated within the system. After logging in, users can enter transportation details, monitor electricity usage, and track carbon emission analytics through graphical dashboards and visual reports. The system also allows users to create sustainability goals, monitor progress continuously, earn achievement badges, and compare environmental performance through leaderboard systems. These features transform sustainability tracking into an interactive and motivating experience rather than a simple calculation process.

The project mainly targets students, working professionals, environmentally conscious individuals, and educational institutions that are interested in sustainability awareness and carbon monitoring. Since climate change and environmental pollution affect all sections of society, the application was designed with a simple and user-friendly interface so that users from different technical backgrounds can use the platform easily. Responsive frontend implementation also improves accessibility across desktop and mobile devices.

The project scope focuses on web-based implementation using manual user inputs for environmental data collection. Users manually provide information related to transportation activities, electricity usage, and lifestyle patterns, which are then processed by the backend system to generate approximate carbon emission values. Although the present implementation uses simplified estimation methods, the overall architecture was designed in a scalable and modular manner to support future enhancements without major structural modifications.

Additional future enhancements may include organization-level sustainability management, multilingual support, cloud deployment, advanced analytics, and government awareness program integration. With these future improvements, the system can evolve into a larger environmental management platform suitable for educational institutions, organizations, and public sustainability initiatives.

1.5 Organization of the Report

This report is organized into five chapters that describe the internship activities, technologies used, implementation methodologies, and learning outcomes related to the Personal Carbon Footprint Application.

Chapter 1: Introduction

This chapter presents the overview of the Personal Carbon Footprint Application including the background of climate change and environmental pollution, problem statement, project objectives, scope of the project, and the importance of sustainability monitoring systems. It also explains the motivation behind developing the application and the role of technology in promoting environmental awareness.

Chapter 2: Company Background

This chapter describes Infosys Limited and the Infosys Springboard Virtual Internship platform. It explains the vision and mission of the organization, technologies and development environment used during the internship, internship learning experience, and future scope of the project. The chapter also discusses how Infosys Springboard supports practical learning and industry-oriented skill development.

Chapter 3: Technology Stack

This chapter explains the major technologies, frameworks, and development tools used during implementation of the Personal Carbon Footprint Application. It includes detailed discussion of React.js, Spring Boot, PostgreSQL, JWT Authentication, Postman, Maven, and GitHub along with their contribution toward frontend

development, backend processing, authentication handling, database management, testing, and project collaboration.

Chapter 4: Technical Training and Assignment

This chapter discusses the practical implementation work and internship activities performed during the project lifecycle. It includes frontend interface development, backend REST API implementation, database integration, authentication handling, dashboard analytics, debugging activities, API testing, version control practices, and collaborative software engineering workflows followed during development of the application.

Chapter 5: Implementation and Contribution

This chapter presents the system interfaces, dashboard modules, sustainability tracking functionalities, project implementation details, individual contribution, learning outcomes, limitations, future scope, and overall internship experience gained during the development of the Personal Carbon Footprint Application.

Chapter 2

COMPANY BACKGROUND

Infosys Limited was founded in 1981 and has grown into one of the world's leading IT service and consulting organizations with operations across multiple countries. Infosys Limited is one of the leading multinational information technology companies headquartered in India. The company is globally recognized for providing software development, business consulting, digital transformation, cloud computing, artificial intelligence, cybersecurity, and enterprise technology solutions across various industries. Infosys has contributed significantly toward technological innovation and digital modernization by helping organizations improve efficiency, scalability, and digital infrastructure through secure and intelligent software systems.

Apart from enterprise software services, Infosys also focuses strongly on education and professional skill development initiatives. One of the major educational programs launched by the company is Infosys Springboard, a digital learning and virtual internship platform designed to provide students with industry-oriented training and practical exposure to modern technologies. The platform offers certification courses, virtual internships, project-based learning programs, and technical training opportunities in domains such as web development, artificial intelligence, cloud computing, cybersecurity, and software engineering.

The Personal Carbon Footprint Application was developed as part of the Infosys Springboard Virtual Internship 6.0 program. The project focuses on helping users monitor and analyze carbon emissions generated through transportation activities, electricity consumption, and daily lifestyle habits. The application was developed using modern full-stack technologies such as React.js, Spring Boot, PostgreSQL, and JWT Authentication while following collaborative software engineering practices and modern development methodologies. The official logo of Infosys Springboard is shown in Fig. 2.1.

Fig. 2.1: Company Logo

The internship program provided practical exposure to frontend-backend integration, REST API development, database management, authentication handling, responsive UI implementation, debugging techniques, and collaborative project workflows. The project also improved understanding of sustainability-focused software solutions and the role of technology in addressing environmental challenges.

2.1 Infosys Springboard Platform

Infosys Springboard is a digital learning and virtual internship platform developed by Infosys with the primary objective of bridging the gap between academic education and industry requirements. The platform was introduced to help students, fresh graduates, and learners improve their technical knowledge, practical implementation skills, and professional capabilities through industry-oriented training programs and project-based learning activities. Infosys Springboard provides a modern digital learning environment where learners can access educational resources, certification courses, hands-on projects, and internship opportunities across multiple technology domains.

The platform offers a wide range of technical courses and training programs related to web development, cloud computing, artificial intelligence, machine learning, cybersecurity, data science, programming languages, software engineering, Internet of Things (IoT), and database management systems. These learning modules are designed according to current industry standards and emerging technology trends, enabling learners to acquire relevant technical skills required in modern software development and IT industries.

One of the major strengths of Infosys Springboard is its focus on practical learning rather than only theoretical concepts. Students are encouraged to work on real-world projects and implementation-based assignments that simulate professional software development environments. Through these practical activities, learners gain exposure to frontend development, backend implementation, REST API communication, database integration, authentication systems, debugging techniques, responsive UI development, and collaborative software engineering workflows.

The platform also promotes innovation, analytical thinking, teamwork, and problem-solving abilities through project development activities and internship programs. Students work collaboratively in teams where responsibilities are divided among frontend, backend, database, and testing modules. This collaborative approach

improves communication skills, project coordination abilities, and understanding of software development lifecycle processes followed in real-world industries.

Infosys Springboard provides virtual internship programs that allow students to gain industrial exposure without geographical limitations. These internships help learners understand modern development methodologies, Agile workflows, project planning, version control systems, API integration, and software deployment concepts. Students also learn how to use modern development tools and frameworks while implementing scalable and secure software applications.

Another important feature of Infosys Springboard is its accessibility and self-paced learning structure. The platform allows learners from different educational backgrounds and skill levels to access technical content and learning resources easily. Video lectures, coding exercises, assessments, project tasks, certification modules, and guided implementation activities are provided systematically to support continuous learning and skill development.

During the Infosys Springboard Virtual Internship 6.0 program, the Personal Carbon Footprint Application was developed as a full-stack sustainability monitoring system. The project provided practical exposure to modern technologies such as React.js, Spring Boot, PostgreSQL, JWT Authentication, GitHub, Postman, and Maven. The internship involved frontend development, backend API implementation, database management, authentication handling, debugging operations, dashboard analytics, and responsive interface design activities.

The platform also emphasizes professional growth and career readiness by helping learners improve technical confidence, practical implementation abilities, debugging skills, collaborative development practices, and software engineering understanding. Through real-time project implementation and guided mentorship, students develop industry-oriented skills that prepare them for future careers in software development and information technology industries.

Overall, Infosys Springboard serves as an important educational and professional development initiative that combines digital learning, practical implementation, collaborative project development, and industry exposure into a single platform. The initiative significantly contributes toward improving technical education and preparing students for real-world software engineering environments through modern technology training and hands-on internship experiences.

2.2 Vision and Mission

Vision:

The vision of Infosys Springboard is to create an inclusive and accessible digital learning ecosystem that empowers students, learners, and young professionals with industry-relevant technical skills, practical implementation knowledge, and professional development opportunities. The platform aims to prepare learners for modern technology-driven industries by providing high-quality educational resources, hands-on project experience, and exposure to real-world software development environments.

Infosys Springboard focuses on encouraging innovation, continuous learning, problem-solving abilities, and technical excellence among students through modern digital education methodologies. The initiative also aims to support skill development in emerging technologies such as artificial intelligence, cloud computing, cybersecurity, data science, software engineering, and full-stack web development while promoting equal learning opportunities for students from diverse educational and social backgrounds.

Mission:

The mission of Infosys Springboard is to bridge the gap between theoretical academic education and practical industry requirements by providing project-based learning, certification programs, technical training modules, and real-world internship experiences using modern technologies and collaborative software engineering workflows.

The platform is committed to helping learners improve their practical implementation skills, analytical thinking, teamwork coordination, and professional readiness through hands-on development activities and guided learning experiences. Infosys Springboard also aims to create a strong foundation in modern software engineering practices by exposing students to frontend development, backend API implementation, database management, authentication systems, cloud technologies, debugging methodologies, and scalable application development processes.

Another important mission of the platform is to encourage learners to solve real-world challenges using technology-driven solutions and innovation-oriented approaches. Through virtual internships, collaborative project development, and practical implementation tasks, Infosys Springboard prepares students for future careers

in software development, information technology, and emerging digital industries while contributing toward technical education and professional growth.

2.3 Technologies and Development Environment

The Personal Carbon Footprint Application was developed using modern full-stack web development technologies and software engineering methodologies. The project implementation focused on creating a responsive, secure, scalable, and user-friendly sustainability monitoring platform capable of handling frontend interaction, backend processing, database management, and secure authentication functionalities efficiently.

React.js was used for frontend interface development and responsive user interface implementation. React.js is a component-based JavaScript library that simplifies frontend development by enabling reusable UI components, dynamic rendering, state management, and efficient user interaction handling. Using React.js, several frontend modules such as login pages, registration systems, carbon tracking dashboards, sustainability analytics pages, leaderboard systems, goal tracking interfaces, notification panels, and achievement badge modules were developed. Responsive layouts and modern UI principles were implemented to improve accessibility and user experience across desktops, tablets, and mobile devices.

Spring Boot was implemented for backend API development and business logic processing. Spring Boot provides a robust framework for developing secure and scalable REST APIs while simplifying backend configuration and dependency management. The backend system handled functionalities such as user authentication, carbon emission calculations, dashboard analytics generation, sustainability goal management, leaderboard processing, notification handling, and database communication. REST APIs were implemented to establish smooth communication between frontend and backend systems through structured request-response mechanisms.

PostgreSQL was used as the relational database management system for structured data storage and backend data management. Database tables and relational schemas were created for storing user information, carbon tracking records, sustainability goals, achievement badges, leaderboard rankings, notifications, and authentication-related data. Proper relational mapping techniques using primary keys and foreign keys were implemented to maintain data consistency, integrity, and organized backend processing.

JWT (JSON Web Token) Authentication was implemented to ensure secure login handling and protected API communication between frontend and backend systems. Token-based authentication mechanisms were used to validate users and restrict unauthorized access to protected resources. Password encryption, token validation, secure session handling, and authorization mechanisms were implemented to improve application security and reliability throughout system usage.

Several additional development tools and software platforms were also used during project implementation. Postman was used for API testing, debugging, authentication validation, and backend response verification. Different HTTP request methods such as GET, POST, PUT, and DELETE were tested regularly to ensure proper API functionality and smooth frontend-backend communication.

GitHub was used for version control, source code management, repository synchronization, and collaborative development activities. GitHub helped maintain organized project workflows through branch management, code integration, version tracking, and collaborative module implementation practices. The platform also improved teamwork coordination and simplified project synchronization among development modules.

Maven was used for dependency management and backend project configuration during Spring Boot development. Maven simplified library integration, project structure management, build automation, and dependency synchronization processes throughout backend implementation activities.

Visual Studio Code and IntelliJ IDEA were used as the primary development environments for frontend and backend coding activities. Visual Studio Code was mainly used for React.js frontend implementation, UI development, component creation, and debugging operations, while IntelliJ IDEA was used for Spring Boot backend development, REST API implementation, database integration, and backend debugging activities.

The project development environment also involved collaborative software engineering practices such as modular implementation, frontend-backend integration, debugging sessions, API testing, code optimization, and project synchronization workflows. These implementation methodologies improved software maintainability, scalability, security, and overall development efficiency throughout the project lifecycle.

The technologies and development environment used during the implementation of the Personal Carbon Footprint Application provided valuable practical exposure to modern full-stack web development, responsive UI implementation, backend architecture design, authentication systems, database management, debugging methodologies, API communication, and collaborative software engineering practices.

2.4 Internship Environment and Learning Experience

The internship environment provided a highly collaborative and practical learning experience throughout the development lifecycle of the Personal Carbon Footprint Application. The internship focused on industry-oriented software development practices, real-time implementation activities, project coordination, and technical skill enhancement using modern full-stack development technologies. The work environment encouraged continuous learning, innovation, teamwork, and problem-solving through hands-on implementation and collaborative project development methodologies.

During the internship period, different project modules such as frontend interfaces, backend APIs, authentication systems, dashboard analytics, sustainability tracking modules, leaderboard systems, notification handling, and database integration were developed systematically using organized workflows and collaborative engineering practices. Responsibilities were divided among team members according to development areas such as frontend implementation, backend processing, database management, and testing operations. This modular development approach improved project organization and helped maintain efficient coordination between different system components.

The internship provided valuable practical exposure to frontend-backend communication and full-stack web application development processes. Frontend interfaces developed using React.js were integrated with Spring Boot backend APIs through REST API communication mechanisms. Different frontend modules such as dashboards, goal tracking systems, leaderboard pages, notification panels, and authentication interfaces were connected with backend services to establish smooth data exchange and dynamic rendering functionalities.

The internship also significantly improved understanding of responsive UI implementation and user-centric design methodologies. Modern frontend development

principles such as reusable components, responsive layouts, form validation, and dynamic dashboard rendering were implemented during the project lifecycle. User interfaces were designed carefully to ensure smooth accessibility across desktops, tablets, and mobile devices while maintaining proper navigation and user experience standards.

Backend development activities provided practical knowledge related to REST API architecture, authentication systems, business logic implementation, and secure backend processing. APIs were developed for carbon emission tracking, sustainability analytics generation, user profile management, notification handling, leaderboard processing, and goal tracking functionalities. Structured API communication mechanisms and proper request-response handling improved backend efficiency and application reliability.

Authentication and security implementation were important aspects of the internship learning experience. JWT Authentication mechanisms were implemented for secure login handling and protected API communication between frontend and backend systems. Password encryption, token validation, secure authorization handling, and protected route management improved understanding of application security practices and modern authentication workflows.

Database integration activities using PostgreSQL improved practical understanding of relational database management systems, database schema design, relational mapping, CRUD operations, and structured data handling. Different database tables were created for users, carbon records, achievements, goals, notifications, and leaderboard systems while maintaining proper relationships using primary keys and foreign keys.

The internship environment also emphasized debugging methodologies, testing practices, and software optimization techniques. Postman was used for backend API testing, authentication validation, request handling verification, and response analysis. Browser developer tools and debugging utilities were used regularly to identify frontend rendering issues, API integration problems, validation errors, and performance-related implementation challenges.

GitHub-based version control and collaborative development workflows played an important role during the internship period. Source code management, repository synchronization, branch handling, module integration, version tracking, and collaborative implementation activities were performed using GitHub repositories.

Team discussions, debugging sessions, project planning activities, and coordinated development practices improved teamwork abilities and understanding of professional software engineering workflows.

The internship also enhanced analytical thinking, troubleshooting skills, time management abilities, and project coordination techniques. Real-world implementation challenges related to frontend-backend integration, authentication validation, database communication, and responsive interface handling improved practical problem-solving capabilities significantly.

Overall, the internship environment provided valuable industrial exposure and practical experience related to full-stack application development, responsive UI implementation, REST API communication, authentication systems, database management, debugging methodologies, collaborative software engineering practices, and sustainability-focused digital solution development. The experience significantly contributed toward technical skill enhancement, professional growth, and understanding of real-world software development environments.

2.5 Future Scope and Innovation

Infosys Springboard strongly emphasizes innovation, continuous learning, and future-oriented technology development through practical implementation methodologies and project-based learning environments. The platform encourages students and learners to explore modern technologies and develop innovative digital solutions capable of addressing real-world challenges effectively. Through virtual internships, technical training programs, and collaborative development activities, learners gain exposure to modern software engineering concepts, emerging technologies, and scalable application development practices.

The Personal Carbon Footprint Application reflects these objectives by integrating sustainability awareness with modern full-stack web development technologies to encourage environmentally responsible behavior through digital monitoring systems. The project demonstrates how software applications can contribute toward solving environmental challenges by helping users monitor carbon emissions, analyze sustainability performance, and adopt eco-friendly lifestyle practices through interactive dashboards, analytical reports, and engagement-based digital features.

The application was designed using scalable and modular software architecture principles so that future enhancements and advanced functionalities can be integrated

easily without major structural modifications. The current implementation focuses mainly on carbon tracking, sustainability analytics, dashboard reporting, leaderboard systems, goal tracking, achievement badges, and secure authentication mechanisms. However, the flexible architecture of the application supports several future improvements and technology integrations.

One possible future enhancement is the integration of Artificial Intelligence (AI)-based sustainability recommendation systems. AI models can analyze user activities, carbon emission patterns, and sustainability performance to generate personalized eco-friendly suggestions and optimized carbon reduction strategies. Machine learning algorithms can also help predict future carbon trends and improve environmental analytics accuracy.

Another important future scope is cloud deployment and cloud-based scalability implementation. Deploying the application on cloud platforms can improve system accessibility, performance, scalability, and real-time data management capabilities. Cloud infrastructure can also support large-scale user management and advanced sustainability analytics for organizations and institutions.

The application can also be enhanced through Internet of Things (IoT)-enabled environmental monitoring systems. IoT sensors and smart devices can automatically collect real-time environmental and energy consumption data instead of relying only on manual user inputs. This integration can improve data accuracy, automation, and real-time sustainability tracking functionalities significantly.

Mobile application integration is another important future enhancement. Developing Android and iOS mobile applications can improve accessibility and allow users to monitor sustainability activities conveniently from smartphones and portable devices. Mobile notifications, reminders, and activity tracking systems can further increase user engagement and environmental awareness.

Additional future improvements may include multilingual support, advanced graphical analytics, organization-level sustainability dashboards, smart energy monitoring systems, government awareness program integration, and community-based environmental collaboration features. These enhancements can transform the application into a comprehensive environmental management and sustainability monitoring platform suitable for educational institutions, organizations, industries, and public awareness initiatives.

The project also demonstrates the growing role of technology in promoting sustainable development and environmental responsibility. Modern software systems can influence user behavior positively when designed with interactive and analytical features that simplify complex environmental information. Through digital sustainability platforms such as the Personal Carbon Footprint Application, users can better understand their environmental impact and make informed decisions regarding eco-friendly practices and carbon reduction activities.

Overall, the Infosys Springboard Virtual Internship provided valuable practical exposure to modern software development technologies, collaborative software engineering workflows, responsive frontend implementation, backend API development, authentication systems, database management, debugging methodologies, and sustainability-focused digital solution development. The experience significantly improved technical knowledge, analytical thinking, implementation skills, teamwork coordination, and understanding of real-world software engineering practices while preparing students for future opportunities in the technology industry.

Chapter 3

TECHNOLOGY STACK

The selection of appropriate technologies plays an important role in modern web application development because the overall performance, scalability, security, maintainability, and user experience of the system depend greatly on the tools used during implementation. Modern full-stack applications require efficient frontend frameworks, reliable backend processing systems, secure authentication mechanisms, and scalable database management solutions to ensure stable functionality and smooth communication between different system layers.

For the development of the Personal Carbon Footprint Application, modern full-stack web technologies were selected to provide responsive frontend design, secure API communication, efficient database management, and scalability for future enhancements. The application was developed using React.js for frontend implementation, Spring Boot for backend API development, PostgreSQL for relational database management, and JWT authentication for secure user access and protected API communication.

This chapter explains the major technologies and development tools used during the implementation of the Personal Carbon Footprint Application and describes their contribution toward frontend development, backend processing, database management, authentication handling, API communication, and overall system functionality.

3.1 React.js

React.js is a popular JavaScript library used for developing dynamic, responsive, and interactive user interfaces for modern web applications [1]. It was developed by Facebook and is widely used in frontend web development because of its component-based architecture, efficient rendering system, and scalability features. React.js allows developers to create reusable UI components, which improves frontend maintainability, reduces repetitive coding, and simplifies large-scale application development

For the Personal Carbon Footprint Application, React.js was used to develop the frontend section of the system. The frontend interface includes user registration pages, login forms, dashboard analytics, lifestyle survey modules, carbon history tracking, sustainability goal management, leaderboard systems, notification panels, Eco Marketplace pages, and profile management interfaces. React.js helped organize these

functionalities into reusable frontend components, making the application more modular, maintainable, and scalable.

Another important feature of React.js is its Virtual DOM rendering mechanism. Instead of refreshing the complete webpage after every user interaction, React.js updates only the required interface components dynamically. This significantly improves frontend performance, dashboard responsiveness, and user interaction speed. The Virtual DOM mechanism also reduces unnecessary rendering operations and creates smoother application behavior during navigation and dashboard analytics updates.

React.js also simplified frontend routing and state management during development. Dynamic sustainability data, dashboard analytics, leaderboard rankings, and user-specific environmental information could be updated efficiently without requiring complete page reloads. The framework supports integration with external libraries and frontend tools, which simplified chart implementation, responsive layouts, and graphical dashboard visualization during frontend development. The frontend development structure implemented using React.js is illustrated in Fig. 3.1.

Fig. 3.1: React.js

Another important reason for selecting React.js was its scalability and strong community support. Since the Personal Carbon Footprint Application may be expanded in the future with advanced analytics, AI-based sustainability recommendations, mobile integration, and additional environmental monitoring features, React.js provides a flexible and scalable frontend foundation for future enhancements.

Responsive frontend implementation was considered an important requirement during system development because users may access the platform through desktops, laptops, tablets, and mobile devices. CSS Flexbox layouts, responsive UI components, and structured frontend design principles were implemented along with React.js to ensure

proper alignment, accessibility, and usability across different screen sizes and device resolutions.

React.js played a major role in developing a modern, responsive, interactive, and user-friendly frontend interface for the Personal Carbon Footprint Application. The framework improved frontend performance, enhanced user experience, simplified interface management, and supported scalable full-stack web application development throughout the internship project.

3.2 Spring Boot

Spring Boot is a Java-based backend development framework used for developing secure, scalable, and production-ready web applications [2]. It is part of the Spring Framework ecosystem and simplifies backend implementation by reducing configuration complexity and providing built-in support for REST API development, dependency management, security integration, and database communication. Spring Boot is widely used in modern full-stack application development because it improves development efficiency, scalability, maintainability, and backend performance.

For the Personal Carbon Footprint Application, Spring Boot was used to develop backend REST APIs and implement core business logic functionalities. The backend system handles important operations such as user authentication, carbon emission calculation processing, dashboard analytics generation, sustainability goal tracking, leaderboard management, achievement handling, notification processing, and database communication. Backend APIs developed using Spring Boot establish communication between the React.js frontend and PostgreSQL database system.

One of the major advantages of Spring Boot is its simplified project configuration and dependency management system. Developers can create backend services efficiently without manually configuring large numbers of XML or server configuration files. This significantly improved development productivity and reduced backend implementation complexity during the internship project.

Spring Boot also provides strong support for REST API development. APIs were developed to process frontend requests related to login authentication, dashboard analytics, carbon tracking operations, sustainability surveys, marketplace management, and leaderboard data retrieval. These APIs enable smooth communication between frontend interfaces and backend processing modules through structured JSON-based

request and response handling. The backend development framework used during implementation is shown in Fig. 3.2.

Fig. 3.2: Spring Boot

Security implementation was another important reason for selecting Spring Boot. The framework supports JWT authentication, role-based authorization, password encryption, validation handling, and secure API communication. These security mechanisms improved application reliability and protected sensitive user information from unauthorized access. Protected APIs require valid authentication tokens before allowing users to access restricted functionalities such as dashboard operations, profile management, and sustainability tracking modules.

The backend implementation using Spring Boot provided a stable, secure, scalable, and efficient processing environment for the Personal Carbon Footprint Application. The framework significantly improved backend organization, API communication, authentication handling, database management, and overall system reliability during the internship project development process.

3.3 PostgreSQL

PostgreSQL is an open-source relational database management system widely used for secure, scalable, and efficient data storage in modern web applications [3]. It supports advanced SQL operations, relational database structures, indexing, transaction management, query optimization, and data integrity mechanisms. PostgreSQL is highly reliable and is commonly used in enterprise-level applications because of its stability, performance, security, and support for complex database operations.

For the Personal Carbon Footprint Application, PostgreSQL was used as the primary database management system for storing application-related information. The database stores user account details, carbon tracking records, sustainability goals,

achievement badges, leaderboard rankings, marketplace transactions, notification records, and authentication-related information. PostgreSQL acts as the central storage system of the application and supports communication between frontend interfaces and backend APIs. One of the major reasons for selecting PostgreSQL was its reliability and scalability.

The database can efficiently manage large amounts of structured relational data and supports future application expansion without major architectural changes. Since the project contains multiple interconnected modules such as users, goals, achievements, notifications, transactions, and leaderboard systems, relational database management was necessary to maintain proper relationships and ensure data consistency across the application.

Database normalization techniques were applied during schema design to reduce data redundancy and improve maintainability. Primary keys and foreign keys were implemented to establish structured relationships between database tables. These relational connections improved data organization, prevented duplicate records, and simplified backend data processing operations. The PostgreSQL database architecture and relational storage structure are illustrated in Fig. 3.3.

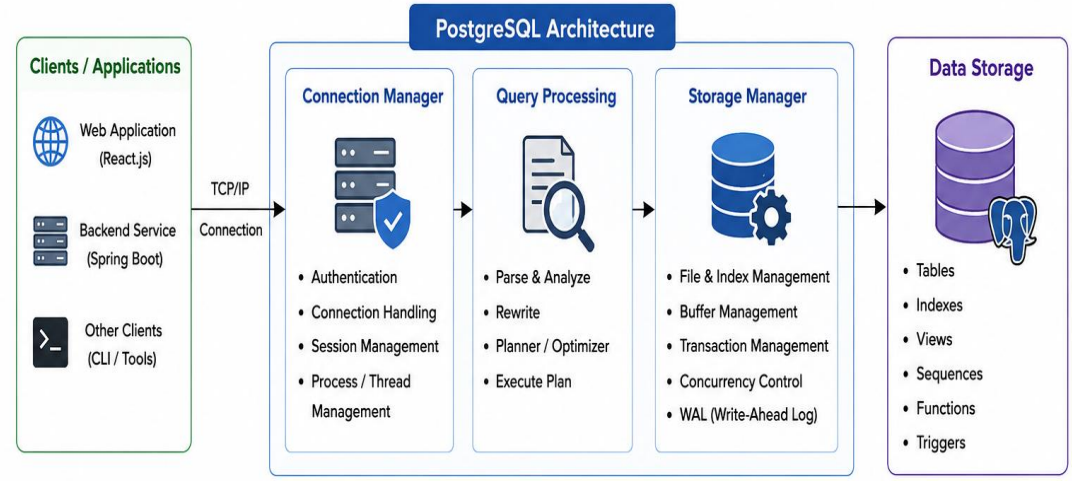


Fig. 3.3: PostgreSQL Database Structure

PostgreSQL also provides strong query optimization and indexing capabilities, which significantly improved dashboard analytics, leaderboard performance, and backend data retrieval speed. Efficient database queries reduced API response time and improved overall system responsiveness during frontend-backend communication. Structured relational storage also simplified reporting operations and historical sustainability tracking.

Another important advantage of PostgreSQL is its strong security and transactional support. The database supports secure authentication integration, access control management, transaction consistency, backup operations, and reliable data validation mechanisms. These features improved overall system reliability and protected sensitive user sustainability records from unauthorized access or data corruption.

PostgreSQL integrates efficiently with Spring Boot backend services using JDBC and ORM-based database communication mechanisms. This integration simplified CRUD operations, query handling, and backend database processing during implementation. Backend APIs developed using Spring Boot communicated with PostgreSQL to retrieve, update, store, and manage sustainability-related information dynamically.

PostgreSQL played an important role in providing secure, scalable, and organized database management for the Personal Carbon Footprint Application. The database system improved data consistency, backend performance, dashboard analytics processing, and reliable storage management throughout the internship project implementation.

3.4 JWT Authentication

JWT (JSON Web Token) authentication is a modern and secure authentication mechanism used for verifying user identity and maintaining authenticated sessions in web applications [11]. In modern full-stack systems, frontend and backend modules communicate continuously through REST APIs, making application security an important requirement during development. JWT authentication is widely used because it provides secure, stateless, and efficient communication between different layers of the application without storing session information on the server side.

For the Personal Carbon Footprint Application, JWT authentication was implemented to secure user login operations and protect backend APIs from unauthorized access. When users log into the system using valid credentials, the backend server verifies user information stored in the PostgreSQL database. After successful verification, the backend generates a secure JWT token and sends it to the React.js frontend application. This token acts as a digital authentication key and is included in future API requests to validate user authorization during communication

between frontend and backend systems. The JWT authentication workflow implemented for secure API communication is shown in Fig. 3.4.

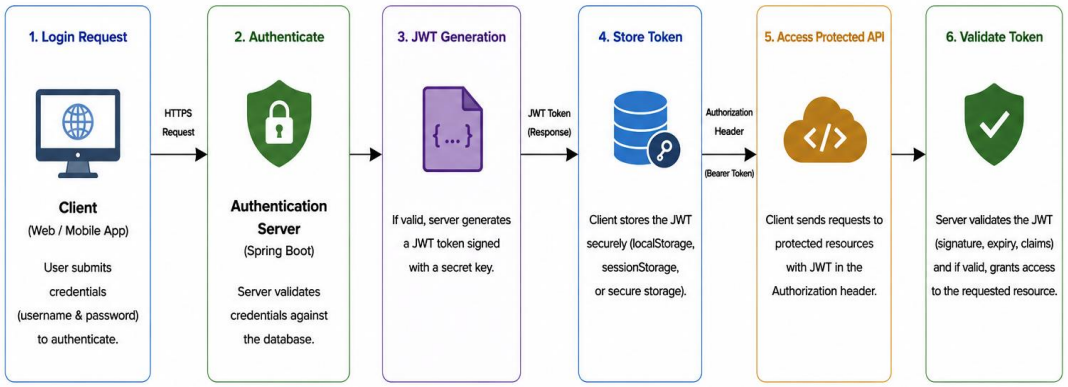


Fig. 3.4: JWT Authentication Process

JWT authentication improved application security. Since tokens are digitally signed and verified during every API request, unauthorized users cannot easily access protected resources or manipulate application operations. Protected APIs such as dashboard analytics, carbon tracking modules, goal management, leaderboard access, profile settings, and marketplace functionalities require valid authentication tokens before processing requests. This improves system reliability, data protection, and user privacy within the application.

Another important advantage of JWT authentication is efficient session management. Traditional session-based authentication systems require storing session information on the server side, which increases backend complexity and memory usage. In contrast, JWT supports stateless authentication, meaning the server does not need to maintain separate session storage for every user. This reduces backend overhead, improves scalability, and enhances application performance, especially in distributed full-stack systems.

JWT authentication also simplified secure communication between React.js frontend interfaces and Spring Boot backend APIs. Authentication tokens are attached with API requests using authorization headers, allowing the backend system to validate user access efficiently before processing operations. This mechanism helped maintain secure communication throughout dashboard access, carbon tracking operations, leaderboard retrieval, and administrative management activities.

Spring Security integration was also implemented along with JWT authentication to improve authorization handling and route protection within the

backend system. Password encryption and validation mechanisms further strengthened account security and reduced risks related to unauthorized access and credential misuse. JWT authentication played an important role in improving application security, protecting sensitive user information, and maintaining secure communication between frontend interfaces and backend APIs throughout the implementation of the Personal Carbon Footprint Application.

3.5 Postman

Postman is a widely used API testing and development tool that helps developers test, analyze, debug, and validate backend services and REST APIs during software development [4]. In modern full-stack web applications, frontend and backend modules communicate continuously through APIs, making API testing an important part of the development process.

For the Personal Carbon Footprint Application, Postman was used extensively during backend development and API testing phases. The backend APIs developed using Spring Boot were tested using Postman before frontend integration to ensure stable communication and proper functionality between frontend interfaces, backend services, and PostgreSQL database systems. APIs related to user registration, login authentication, dashboard analytics, carbon emission calculations, sustainability goal tracking, leaderboard retrieval, achievement management, notifications, and marketplace operations were verified using different HTTP request methods and response validation mechanisms. API testing and response validation using Postman are illustrated in Fig. 3.5.

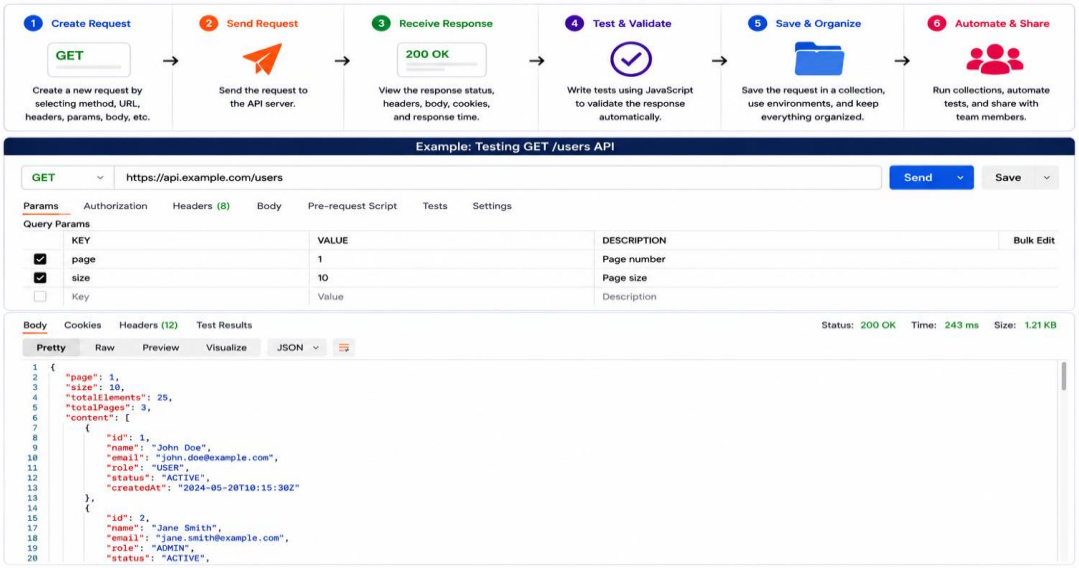


Fig. 3.5: API Testing Using Postman

Another advantage of using Postman was its ability to simplify backend debugging and API verification. During implementation, Postman helped identify several issues such as incorrect API responses, authentication failures, validation errors, response delays, improper request handling, and data transfer problems between frontend and backend systems. By testing APIs independently before frontend integration, synchronization issues were reduced and overall application stability improved significantly.

Postman also played an important role in testing JWT authentication and protected API endpoints. Authentication tokens generated during login operations were validated using secured API requests to ensure proper authorization handling. This helped verify that unauthorized users could not access protected backend resources without valid authentication tokens. Protected APIs related to dashboard analytics, profile management, sustainability goals, and leaderboard systems were tested thoroughly using authenticated requests.

Different HTTP request methods such as GET, POST, PUT, and DELETE were tested during backend implementation to validate CRUD operations and ensure proper API functionality. Request headers, request bodies, JSON responses, status codes, and validation rules were analyzed carefully to verify correct backend behavior.

Another important advantage of Postman was its support for structured API response analysis. JSON responses returned from backend services could be analyzed efficiently, allowing developers to validate response structures, authentication handling, error management mechanisms, and database communication results. This improved debugging efficiency, simplified backend testing, and reduced implementation complexity during the development process.

Postman played an important role in verifying backend API functionality, improving debugging efficiency, validating secure authentication mechanisms, and ensuring stable communication between frontend interfaces and backend services during the implementation of the Personal Carbon Footprint Application.

3.6 Maven

Maven is a popular build automation and dependency management tool primarily used for Java-based software development projects [5]. It helps developers manage project libraries, configure application dependencies, automate build processes, and maintain organized project structures efficiently. In modern backend development, especially in

Spring Boot applications, Maven plays an important role in simplifying dependency management, reducing configuration complexity, and improving development productivity.

For the Personal Carbon Footprint Application, Maven was used along with Spring Boot to manage backend dependencies and automate project build operations. During backend development, the system required multiple external libraries and frameworks related to authentication handling, database integration, REST API communication, security configuration, validation processing, and JWT implementation. Instead of manually downloading and configuring each library separately, Maven automatically handled dependency management through the project configuration file known as pom.xml. The dependency management workflow using Maven is illustrated in Fig. 3.6.

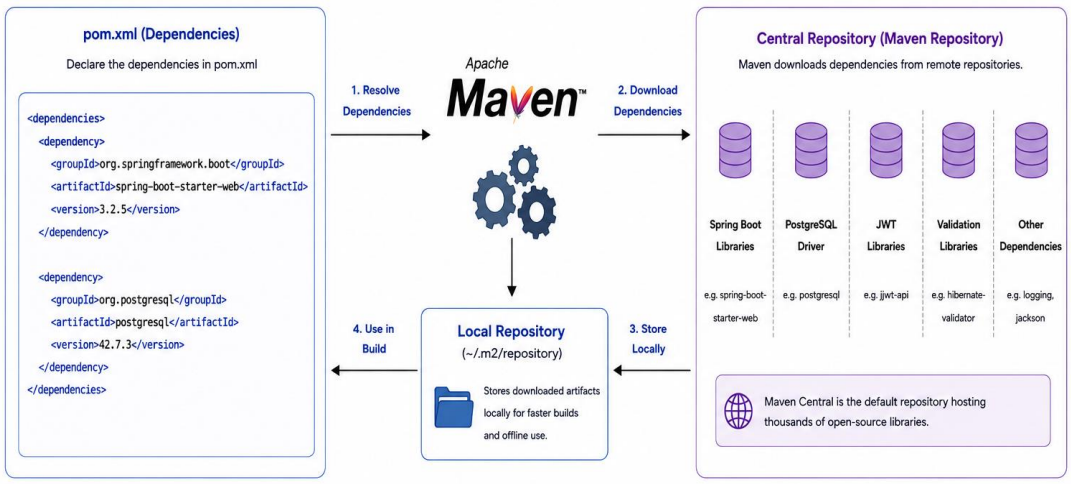


Fig. 3.6: Maven Dependency Management

Maven was used to manage dependencies related to Spring Security configuration, PostgreSQL database connectivity, REST API development, JWT authentication handling, validation frameworks, backend processing modules, and API communication libraries. All required dependencies were defined inside the Maven configuration file, and Maven automatically downloaded compatible library versions from centralized repositories. This reduced manual configuration efforts and significantly improved backend development efficiency during implementation.

One of the major advantages of Maven is its standardized project structure and organized dependency management system. Maven maintains a proper folder hierarchy for source code, configuration files, dependencies, compiled classes, and testing

modules. This organized structure improved backend maintainability, simplified debugging, and reduced project management complexity during development.

Maven also simplified project build automation and dependency updates. Developers could compile backend code, package application files, run testing modules, and manage dependencies using simple Maven commands. This improved development workflow and reduced repetitive manual operations during backend implementation.

Another important advantage of Maven is its compatibility with Spring Boot applications and integrated development environments such as IntelliJ IDEA and Visual Studio Code. Maven integration simplified backend configuration, dependency synchronization, and application deployment processes. The tool also improved project scalability because new libraries and frameworks could be integrated efficiently without major structural modifications.

Maven played an important role in improving backend development efficiency, dependency management, build automation, and project organization during the implementation of the Personal Carbon Footprint Application. The tool simplified backend configuration processes and supported stable, scalable, and maintainable full-stack application development throughout the internship project.

3.7 GitHub

GitHub is a widely used web-based platform designed for version control, source code management, and collaborative software development [12]. It is built on Git, a distributed version control system that helps developers track code changes, manage project versions, and collaborate efficiently within development teams. GitHub provides cloud-based repositories where developers can store, update, organize, and manage project files securely. In modern software development, GitHub plays an important role in teamwork coordination, version tracking, project management, and secure code collaboration.

For the Personal Carbon Footprint Application, GitHub was used for source code management, version control, collaboration, and project synchronization throughout the development process. The frontend and backend source code developed using React.js and Spring Boot was maintained in GitHub repositories to ensure organized project management and efficient collaboration between team members. Different project modules such as frontend interfaces, backend APIs, authentication systems, database integration, and dashboard functionalities were managed using

GitHub repositories and branches. The collaborative version control workflow using GitHub is shown in Fig. 3.7.

Fig. 3.7: GitHub Collaboration and Version Control

One of the major advantages of GitHub is its efficient version control system. Developers can track every modification made to the project source code and restore previous versions whenever necessary. During development, GitHub helped maintain multiple project versions and efficiently track frontend updates, backend modifications, API integrations, and database-related changes. This reduced the risk of code loss and improved development reliability significantly.

GitHub also simplified team collaboration and project coordination. Multiple developers could work on different modules simultaneously without affecting the main project source code. Branching and merging functionalities helped developers implement new features, test functionalities, fix bugs, and integrate modules efficiently. Pull requests and code review mechanisms improved code quality and helped maintain organized development practices throughout the internship project.

Another important advantage of GitHub is its cloud-based repository management and backup support. Project files stored on GitHub remain securely accessible from different systems and locations. This improved project accessibility, data security, and development continuity during implementation. GitHub also supports issue tracking, documentation management, project organization, and workflow monitoring, which helped improve project management efficiency.

GitHub integration with development tools such as Visual Studio Code and IntelliJ IDEA further simplified code synchronization and project updates. Developers could push local changes to remote repositories, retrieve updated project files, and maintain synchronization between team members efficiently. GitHub also improved deployment preparation and project scalability because large codebases and multiple modules could be managed systematically.

Chapter 4

TECHNICAL TRAINING AND ASSIGNMENT

During the internship period, the development team analyzed the requirements and functionalities needed for building the Personal Carbon Footprint Application. The training phase focused on understanding sustainability monitoring concepts, frontend-backend communication, database integration, secure authentication mechanisms, and responsive web application development. Existing carbon footprint tracking platforms were studied to identify limitations and improve user engagement through dashboard analytics, goal tracking, badges, leaderboard systems, and graphical sustainability monitoring features.

The internship mainly emphasized practical implementation of modern full-stack development technologies and software engineering concepts. Real-time development activities such as frontend interface creation, backend API integration, authentication handling, database management, debugging, testing, and version control were performed throughout the internship period. These activities provided hands-on experience related to software development lifecycle processes, collaborative development methodologies, and scalable web application implementation. The overall internship project development workflow followed during implementation is illustrated in Fig. 4.1.

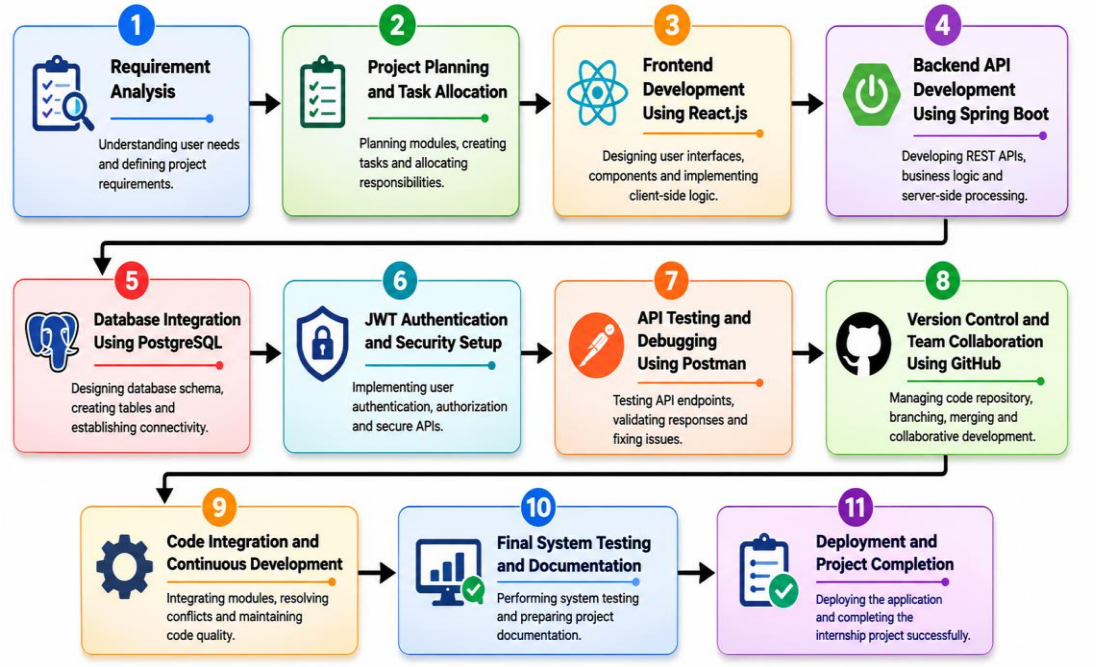


Fig. 4.1: Internship Project Development Workflow

4.1 Internship Activities

During the internship period, several practical development activities were performed for implementing the Personal Carbon Footprint Application. The internship work mainly focused on frontend development, backend API implementation, database connectivity, dashboard analytics, authentication handling, debugging, and system testing using modern full-stack technologies.

The initial phase of the internship involved requirement analysis and project planning activities. The development team studied existing sustainability monitoring platforms and analyzed environmental tracking requirements needed for implementing an interactive carbon footprint management system. Functionalities such as carbon tracking, sustainability goals, dashboard analytics, achievements, leaderboard systems, notifications, and Eco Marketplace integration were planned during this phase.

Frontend development activities were performed using React.js. Responsive interfaces such as login pages, registration modules, dashboards, carbon tracking pages, survey forms, goal tracking interfaces, notification panels, and leaderboard systems were developed during implementation. Reusable frontend components and responsive layouts were designed to improve user experience and accessibility across desktops, tablets, and mobile devices.

Backend development and API integration activities were implemented using Spring Boot. REST APIs were developed for user authentication, carbon emission calculations, sustainability goal processing, leaderboard management, notification handling, profile management, and dashboard analytics generation. Backend services were integrated with frontend interfaces using structured API communication mechanisms.

Database management activities were performed using PostgreSQL. Database tables and relational structures were created for users, carbon records, achievements, goals, badges, notifications, and leaderboard systems. Proper relational mapping techniques using primary keys and foreign keys were implemented to maintain data consistency and organized backend processing.

Authentication and security implementation activities were also performed during the internship period. JWT authentication mechanisms, secure login handling, password encryption, token validation, and protected API communication were

implemented to improve application security and prevent unauthorized access to restricted resources.

Testing and debugging activities were performed using Postman and browser developer tools. Backend APIs, authentication systems, CRUD operations, validation handling, and frontend-backend communication were tested regularly to identify implementation issues and improve overall system stability.

The internship activities provided valuable industrial exposure and practical experience related to full-stack application development, responsive frontend implementation, backend API architecture, authentication handling, database management, debugging techniques, testing methodologies, and collaborative software engineering practices.

4.2 Skills Learned During Training

During the internship training period, several technical and professional skills were learned and improved through practical implementation activities and collaborative software development processes. The internship provided valuable hands-on experience related to modern web technologies, real-time project development, debugging, testing, API communication, database integration, and software engineering workflows.

The following technical skills were practiced and improved during the internship period:

- Frontend Development using React.js
- Backend API Development using Spring Boot
- REST API Communication and Integration
- Database Design and Management using PostgreSQL
- JWT Authentication and Authorization Handling
- Responsive UI Design and Frontend Routing
- Dashboard Analytics and Graphical Visualization
- API Testing and Debugging using Postman
- Version Control and Collaboration using GitHub
- Dependency Management using Maven
- CRUD Operation Implementation
- Form Validation and Error Handling
- Component-Based UI Development

- Backend Security Configuration
- Database Connectivity and Query Handling

In addition to technical skills, several professional skills were also improved during the internship period:

- Team Collaboration and Communication
- Problem-Solving and Debugging Skills
- Project Planning and Task Management
- Software Development Lifecycle Understanding
- Real-Time Development Experience
- Time Management and Coordination
- Analytical Thinking and Troubleshooting
- Collaborative Development Workflow

The internship significantly improved practical understanding of full-stack web application development and provided exposure to industry-oriented software engineering practices. The training activities enhanced confidence in implementing scalable, secure, and responsive applications using modern development technologies and collaborative implementation methodologies.

4.3 Tasks Performed During Internship

During the internship period, several practical development and implementation tasks were performed for the successful development of the Personal Carbon Footprint Application. The internship activities mainly focused on frontend interface development, backend API integration, database management, authentication handling, dashboard implementation, and testing operations using modern full-stack web technologies.

One of the major tasks performed during the internship was frontend development using React.js. Multiple responsive user interfaces such as login pages, registration forms, dashboard modules, carbon tracking interfaces, leaderboard pages, badge systems, Eco Marketplace pages, and notification panels were developed during implementation. Reusable frontend components and responsive layouts were designed to enhance user experience and maintain smooth accessibility across desktops, tablets, and mobile devices.

Backend development activities were also performed using Spring Boot. REST APIs were implemented for handling user authentication, carbon emission calculations,

dashboard analytics, goal tracking, leaderboard processing, notification handling, and profile management functionalities. Backend APIs were integrated with frontend modules to establish smooth communication between different system layers.

Database-related tasks were performed using PostgreSQL. Database tables were created for user records, carbon tracking data, sustainability goals, badges, leaderboard systems, notifications, and authentication handling. Proper relational database structures using primary keys and foreign keys were implemented to maintain data consistency and improve database organization.

JWT authentication implementation was another important internship task. Secure login handling, token generation, authorization validation, and protected API communication were implemented to enhance application security and prevent unauthorized access to protected resources.

API testing and debugging activities were performed using Postman. Different HTTP request methods such as GET, POST, PUT, and DELETE were tested to validate backend functionality and API communication. Authentication handling, response validation, and database communication were verified during backend testing operations.

Version control and project collaboration tasks were also performed using GitHub. Project source code management, version tracking, repository synchronization, and module integration activities were handled using GitHub repositories and branching mechanisms. Team collaboration activities improved development coordination and simplified project management throughout the internship period.

4.4 Tools Practiced During Internship

During the internship training period, several modern software development tools and technologies were practiced for implementing the Personal Carbon Footprint Application. These tools helped improve technical understanding, development efficiency, debugging capability, project organization, and practical full-stack development skills.

React.js was practiced for frontend interface development and responsive UI implementation. The framework helped in understanding component-based architecture, frontend routing, state management, and dynamic dashboard visualization techniques.

Spring Boot was practiced for backend API development and business logic implementation. REST API creation, authentication handling, request validation, backend processing, and secure API communication were implemented using Spring Boot frameworks and libraries.

PostgreSQL was practiced for relational database management and backend data storage. Database table creation, SQL queries, relational mapping, CRUD operations, and structured data handling were implemented during backend integration activities.

JWT authentication mechanisms were practiced to improve application security and secure communication between frontend and backend systems. Token generation, authorization handling, password encryption, and protected API validation were implemented during development.

Postman was practiced for API testing, debugging, and backend verification. HTTP request handling, response analysis, authentication testing, and CRUD operation validation were performed using Postman during implementation and testing phases.
GitHub was practiced for version control, source code management, repository synchronization, and team collaboration activities. Branch management, code integration, commit handling, and project organization were managed using GitHub repositories.

Maven was practiced for dependency management and backend build automation. Maven simplified backend configuration, dependency synchronization, and project structure management during Spring Boot development.

Visual Studio Code and IntelliJ IDEA were also used during implementation for frontend coding, backend development, debugging, API integration, and project management activities.

The practical usage of these tools improved technical knowledge related to modern full-stack development, secure application implementation, collaborative software engineering, API communication, and scalable web application architecture.

4.5 Team Collaboration and Learning Experience

The internship period provided valuable practical exposure to teamwork, project coordination, software development practices, and collaborative problem-solving activities. The development of the Personal Carbon Footprint Application involved

coordination between frontend, backend, and database implementation tasks, which improved communication and teamwork skills during the internship.

Different project modules were divided among team members according to development responsibilities. Frontend-related tasks such as dashboard implementation, goal tracking interfaces, leaderboard pages, and responsive UI development were coordinated with backend API implementation and database integration activities. Team discussions and collaborative planning helped maintain synchronization between different modules throughout the project lifecycle.

During development, several implementation challenges related to API integration, authentication handling, frontend responsiveness, database communication, and debugging were encountered. Problem-solving activities and collaborative discussions helped identify implementation issues and enhance overall system stability. Debugging backend APIs, resolving authentication validation errors, correcting frontend integration problems, and optimizing dashboard functionality improved practical troubleshooting skills significantly.

The internship experience significantly improved practical knowledge related to full-stack web development, REST API communication, responsive frontend implementation, backend processing, database management, authentication security, and project coordination. It also enhanced confidence in implementing real-world software applications using modern technologies and collaborative development methodologies.

4.6 System Architecture

The Personal Carbon Footprint Application follows a client-server architecture where the frontend, backend, and database operate as separate layers. This architecture improves scalability, modularity, and maintainability because each layer performs independent responsibilities while communicating through APIs.

The frontend layer handles user interaction and graphical presentation. Users interact with forms, dashboards, charts, and profile pages developed using React.js. The frontend collects user inputs and sends requests to backend APIs for processing.

The backend layer acts as the core processing unit of the application. Spring Boot APIs handle user authentication, carbon calculations, goal tracking, leaderboard processing, and database operations. The backend validates user requests, performs calculations, and sends responses back to the frontend.

The database layer stores all application-related information such as user details, carbon tracking records, goals, achievements, and leaderboard data. PostgreSQL was selected because it supports efficient relational database management and secure data handling.

After processing requests, the backend sends responses back to the frontend where data is displayed using graphical dashboards and visual analytics. This modular architecture simplifies future expansion because new modules can be added without affecting the complete system structure.

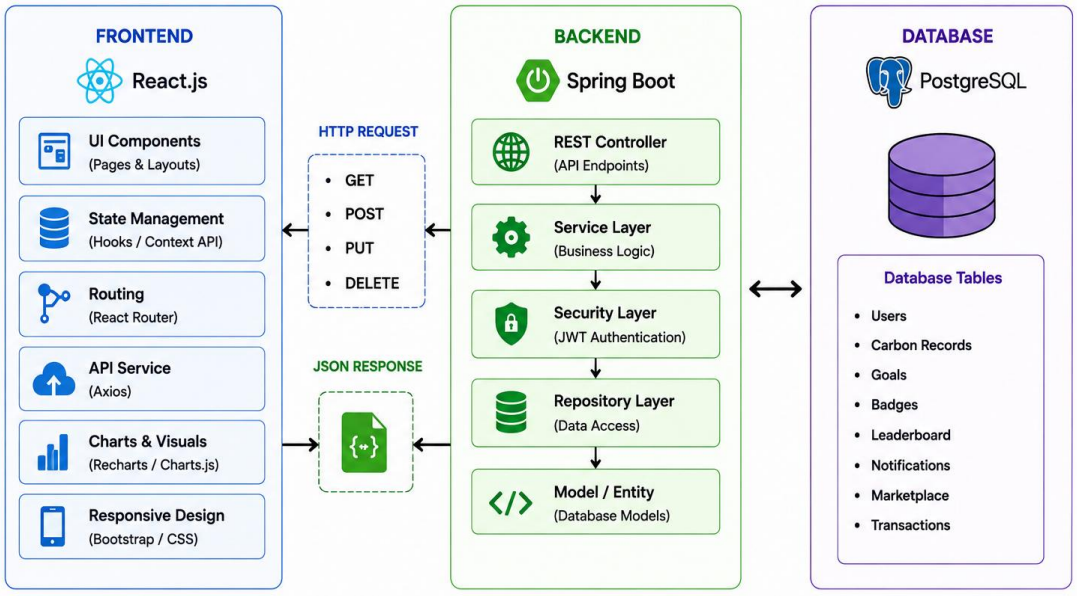


Fig. 4.2: System Architecture

4.7 Frontend Design and Implementation

The frontend section of the Personal Carbon Footprint Application was developed using React.js with the objective of creating a responsive, interactive, and user-friendly interface that simplifies sustainability monitoring and improves user engagement. In modern web applications, the frontend plays an important role because it acts as the communication layer between users and the backend system. A well-designed frontend interface improves accessibility, enhances usability, and helps users interact with system functionalities more efficiently.

The frontend architecture of the application was designed using React.js component-based development methodology. This approach allows the application interface to be divided into smaller reusable UI components, improving maintainability, scalability, and development efficiency. Different modules such as registration forms, login interfaces, dashboard analytics, sustainability goal tracking pages, leaderboard

systems, achievement sections, notification panels, and profile management interfaces were implemented as separate frontend components. Reusable component structures reduced repetitive coding and simplified frontend maintenance during development.

Responsive design was considered an important requirement during frontend implementation because users may access the application from different devices such as desktops, tablets, and smartphones. CSS styling techniques, Flexbox layouts, and responsive UI structures were implemented to ensure that the application interface automatically adjusts according to different screen sizes and resolutions. This responsive behavior improves accessibility and provides consistent user experience across multiple platforms.

Special attention was given to dashboard visualization and analytical presentation because environmental monitoring data becomes easier to understand through graphical representation rather than complex numerical outputs. Interactive charts, progress indicators, emission summaries, activity reports, and sustainability analytics were integrated into the frontend interface to improve readability and user understanding. Graphical visualization also enhances user engagement and encourages continuous interaction with sustainability tracking modules.

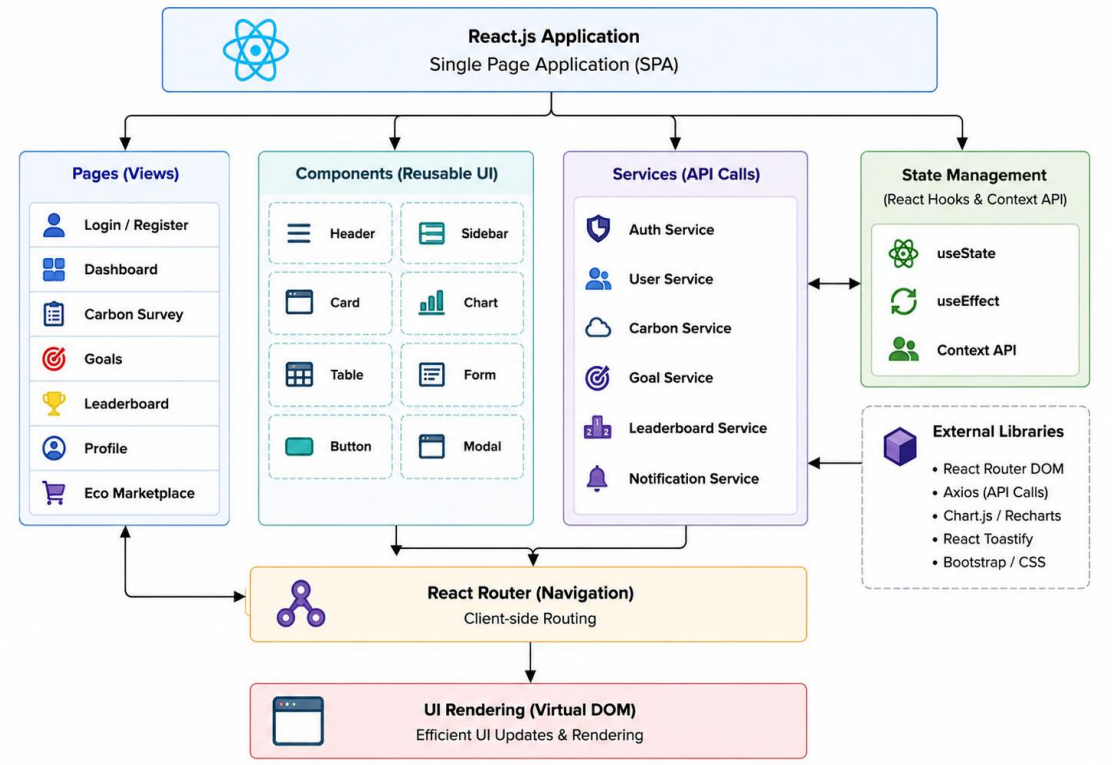


Fig. 4.3: Frontend Workflow

Frontend navigation and interface organization were designed carefully to improve usability and reduce operational complexity. Users can easily move between modules such as dashboards, carbon tracking pages, goals, achievements, leaderboard systems, and profile management interfaces through organized navigation menus and structured layouts. Simplified navigation improves overall user interaction and reduces confusion while using the application. Form validation mechanisms were also implemented within frontend interfaces to improve data accuracy and prevent invalid submissions. Validation checks ensure that users enter proper information before requests are processed by backend APIs. Error messages and input restrictions improve user experience and reduce incorrect data entry during registration, login, and sustainability survey operations.

React.js also improved frontend performance through its virtual DOM rendering mechanism. Instead of refreshing the complete page after every interaction, React.js updates only required interface components dynamically. This improves dashboard responsiveness, reduces unnecessary rendering operations, and creates smoother user interaction throughout the application. The frontend implementation focused on developing an engaging, responsive, and visually understandable sustainability monitoring platform that improves accessibility, simplifies environmental tracking, and enhances user experience through modern web interface design principles.

4.8 Backend Design and Implementation

The backend section of the Personal Carbon Footprint Application was implemented using Spring Boot, which provided a secure, scalable, and efficient environment for handling business logic, API communication, authentication processing, and database management. In full-stack web applications, the backend acts as the core processing layer responsible for handling system operations, validating user requests, managing application logic, and maintaining communication between frontend interfaces and the database system.

The backend architecture was designed using layered implementation methodology to improve modularity, maintainability, and organized system structure. The application backend was divided into multiple layers such as the Controller Layer, Service Layer, Repository Layer, and Security Layer. Each layer performs specific responsibilities, reducing system complexity and improving development organization.

The Controller Layer is responsible for handling incoming API requests from frontend components. Whenever users perform activities such as login authentication, dashboard access, goal creation, carbon tracking, or leaderboard retrieval, frontend requests are transferred to backend controllers through REST APIs. The controller layer validates incoming requests and forwards them to appropriate service modules for further processing.

The Service Layer contains the core business logic implementation of the application. Important functionalities such as carbon emission calculations, sustainability goal tracking, leaderboard ranking algorithms, achievement handling, dashboard analytics generation, and environmental data processing were implemented within this layer. Separating business logic from API handling improves code maintainability and simplifies future modifications.

The Repository Layer manages communication between backend services and the PostgreSQL database. Database operations such as data insertion, retrieval, updates, and deletion were implemented through structured repository methods and query handling mechanisms. This layer simplifies relational database operations and improves data management efficiency throughout the application.

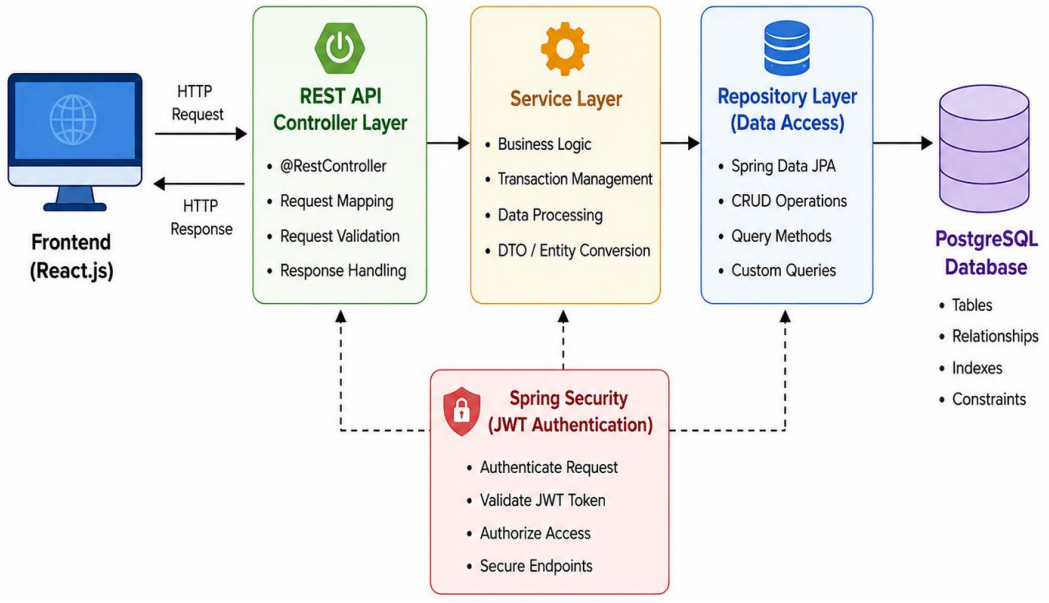


Fig. 4.4: Backend Workflow

Security implementation was treated as an important aspect of backend development because the system stores personal user information and sustainability tracking records. JWT-based authentication and authorization mechanisms were implemented to secure backend APIs and protect user sessions. Protected routes require

valid authentication tokens before granting access to restricted functionalities. Password encryption and secure token validation further improved application security and protected sensitive user information from unauthorized access.

Exception handling and validation mechanisms were also implemented within backend modules to improve system reliability and reduce runtime failures. Invalid requests, authentication failures, incorrect inputs, and database-related issues are handled using structured exception management techniques. This improves application stability and prevents unexpected system crashes during operation. Spring Boot significantly simplified backend implementation because it provides built-in support for REST API development, dependency management, security configuration, database integration, and scalable application architecture. The framework reduced manual configuration complexity and improved development productivity during implementation.

REST API communication was another major component of backend implementation. APIs were developed to establish smooth communication between React.js frontend interfaces and PostgreSQL database operations. JSON-based request and response structures were used to exchange data efficiently between different system layers. Proper API integration improved modularity, simplified frontend and backend interaction, and supported scalable application development. The backend implementation provided a stable, secure, and scalable processing environment for the Personal Carbon Footprint Application. The modular architecture, secure authentication handling, organized API communication, and efficient database integration collectively improved system reliability, maintainability, and overall application performance.

Chapter 5

IMPLEMENTATION AND CONTRIBUTION

During the internship period, multiple practical development and verification activities were performed to ensure smooth functionality of the Personal Carbon Footprint Application. The internship work involved frontend interface development, backend API integration, authentication handling, dashboard implementation, debugging, and database connectivity. Different modules of the application were verified regularly to improve system stability, usability, and user experience across different devices and browsers.

5.1 System Interfaces Developed During Internship

This section describes the major user and administrative interfaces developed for the Personal Carbon Footprint Application. These interfaces were designed to provide smooth interaction between users and the system while maintaining usability, responsiveness, and accessibility across different devices. The frontend interfaces were implemented using React.js, while backend communication was handled through Spring Boot REST APIs and PostgreSQL database integration.

The application contains multiple interconnected modules that support carbon footprint tracking, sustainability monitoring, user management, gamification, and administrative control functionalities. Each interface was developed with a user-friendly layout to simplify navigation and improve overall user experience. Responsive UI components, form validation mechanisms, secure authentication handling, and dashboard visualizations were integrated throughout the application to improve functionality and usability.

The user-side modules mainly focus on environmental tracking activities such as lifestyle surveys, carbon analytics, sustainability goals, badges, leaderboard rankings, Eco Marketplace access, and notification management. These modules help users monitor their environmental impact and encourage sustainable lifestyle improvements through interactive dashboard systems and graphical analytics. The administrative modules provide management and monitoring functionalities required for maintaining system operations. Administrator interfaces support user management, badge creation, notification handling, transaction monitoring, admin activity logs, and system configuration management. These modules help maintain platform security, organizational control, and efficient sustainability monitoring across the application.

The following subsections provide detailed explanations and screenshots of the major system interfaces developed during the implementation of the Personal Carbon Footprint Application.

5.1.1 Home Page Interface

The Home Page acts as the main entry point of the Personal Carbon Footprint Application. It introduces users to the CarbonCalc platform and provides an overview of the major functionalities available within the system. The interface was designed with a clean and responsive layout to ensure better accessibility and user engagement across desktop and mobile devices.

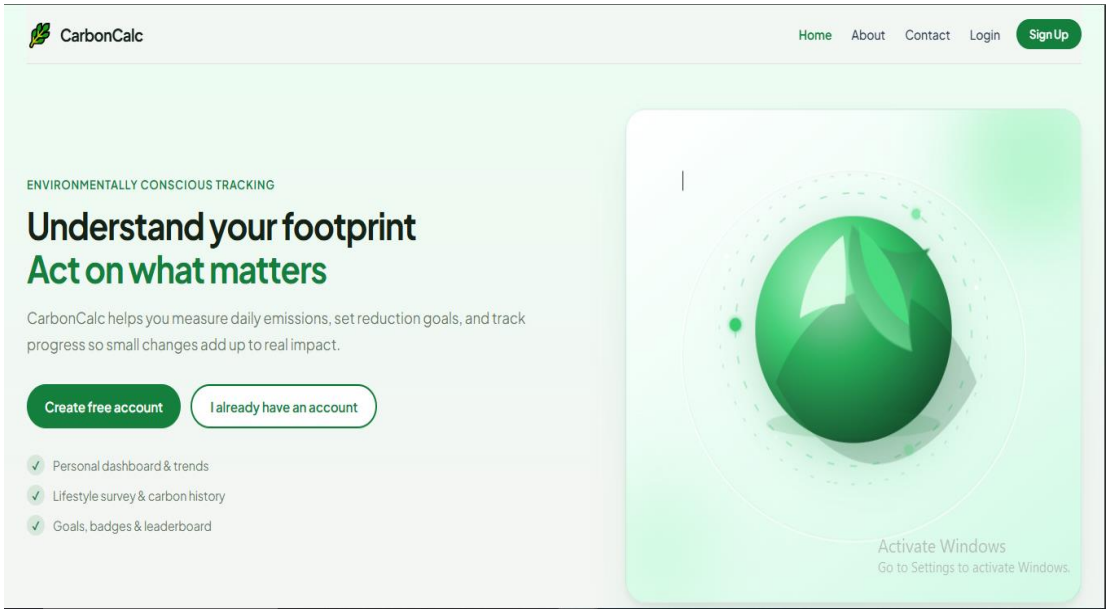


Fig. 5.1: Home Page Interface

The home page highlights important modules such as carbon footprint tracking, sustainability goal management, dashboard analytics, Eco Marketplace access, and environmental awareness features. Navigation menus are provided for easy access to login, registration, and dashboard sections. Informative banners and sustainability-related graphics improve visual appearance and help users understand the purpose of the application clearly. Fig. 5.1 shows the responsive home page interface developed using React.js and integrated with sustainability awareness features.

The landing page also promotes eco-friendly awareness by displaying motivational sustainability content and platform objectives. Responsive frontend implementation using React.js ensures smooth interaction and proper alignment of interface elements across different screen sizes.

5.1.2 User Authentication Module

5.1.2.1 Login Page

The Login Page allows registered users to securely access the system using their authentication credentials. Users enter their email ID or username along with password details to access dashboard functionalities and sustainability tracking modules. The secure login interface developed for user authentication and account access is shown in Fig. 5.2.

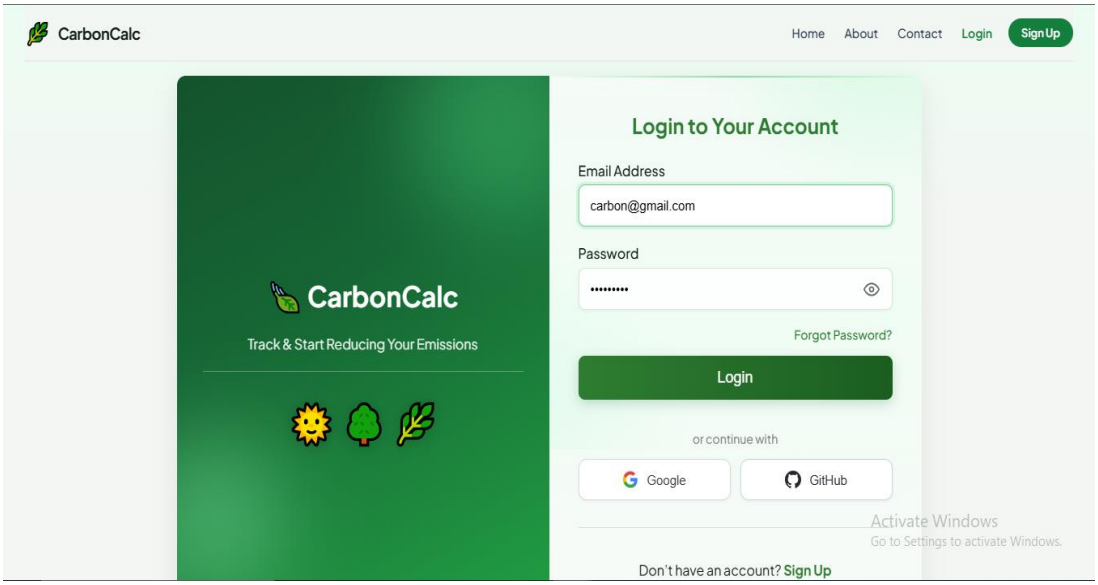


Fig. 5.2: Login Page Interface

JWT authentication was implemented to ensure secure login handling and protected API communication. Form validation mechanisms prevent invalid submissions and improve security by restricting unauthorized access attempts. Proper error handling messages are displayed whenever incorrect login credentials are entered. Fig. 5.2 demonstrates the secure login system integrated with JWT authentication and backend validation mechanisms. The login interface was designed using responsive frontend components to maintain usability and accessibility across multiple devices.

5.1.2.2 Registration Page

The Registration Page allows new users to create accounts within the CarbonCalc platform. Users provide personal details such as username, email address, and password during account creation. Password encryption and validation mechanisms improve account security and prevent weak credential usage. The user registration interface designed for new account creation is illustrated in Fig. 5.3.

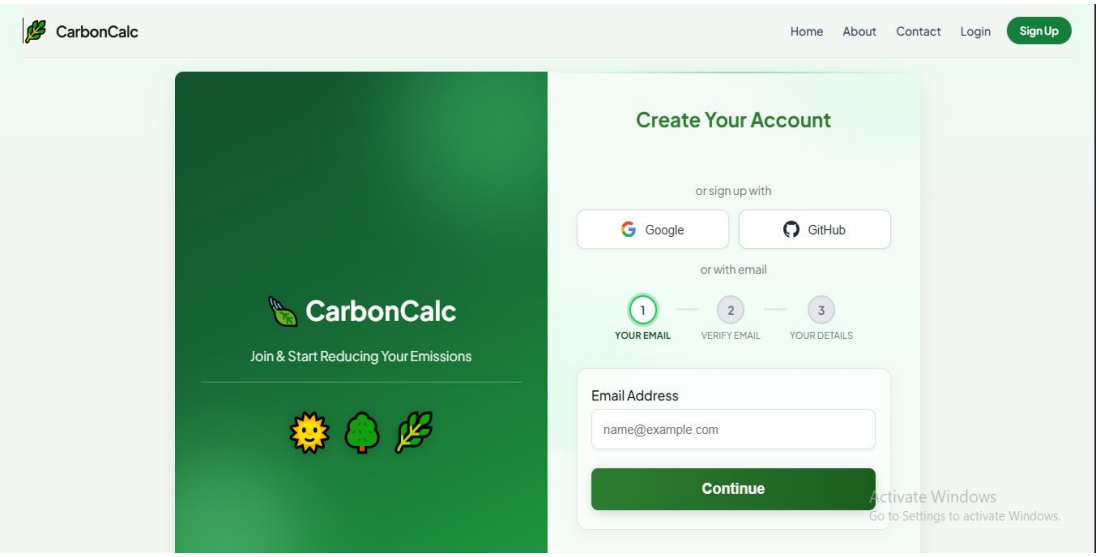


Fig. 5.3: Registration Page Interface

The registration module was implemented using React.js forms integrated with Spring Boot backend APIs for secure data storage and account management. Successful registration enables users to access sustainability tracking features, dashboard analytics, goals, and leaderboard systems. Fig. 5.3 illustrates the user registration interface connected with backend APIs and database storage operations. The authentication module overall improves platform security, user management, and personalized environmental monitoring.

5.1.3 User Dashboard Module

The Dashboard Module provides a centralized overview of user sustainability performance and carbon emission analytics. It displays category-wise carbon emissions related to transportation, electricity usage, and lifestyle activities using graphical charts and visual summaries. The dashboard analytics interface displaying sustainability summaries and carbon tracking information is shown in Fig. 5.4.

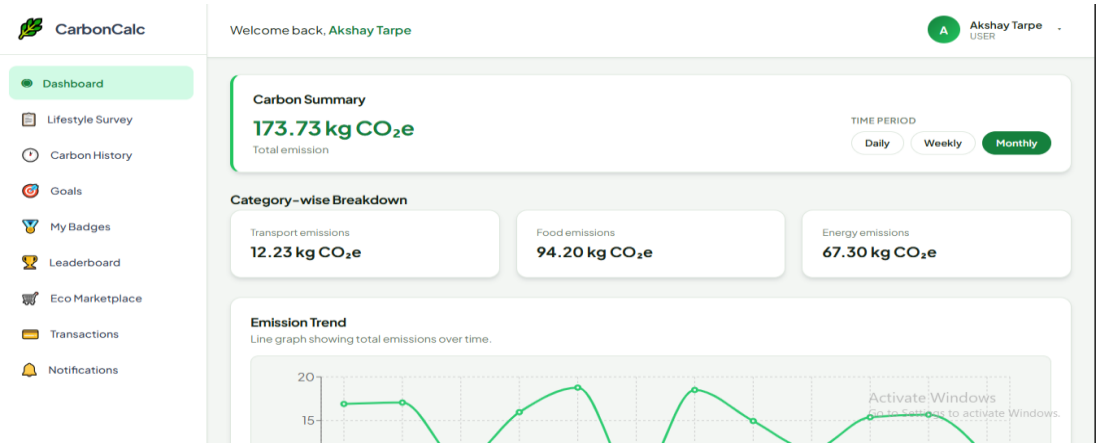


Fig. 5.4: User Dashboard Interface

Interactive dashboard components help users monitor environmental impact efficiently through visual analytics instead of complex numerical outputs. Progress indicators, activity summaries, and emission trend charts improve user understanding and engagement. Fig. 5.4 presents the dashboard analytics module displaying graphical carbon emission summaries and sustainability statistics. The dashboard interface was developed using React.js charts and responsive UI components integrated with backend REST APIs for real-time data retrieval and analytics processing.

5.1.4 Lifestyle Survey Module

The Lifestyle Survey Module allows users to submit sustainability-related information required for carbon footprint calculations. Users enter details related to transportation methods, food consumption habits, electricity usage, and environmental lifestyle activities. The backend system processes this information using predefined environmental emission factors to estimate approximate carbon emissions. The survey module helps users understand how daily habits contribute toward environmental pollution and climate change. Input validation and structured forms improve data accuracy and simplify user interaction during survey submission. The sustainability goal management interface developed for progress monitoring and target tracking is shown in Fig. 5.5.

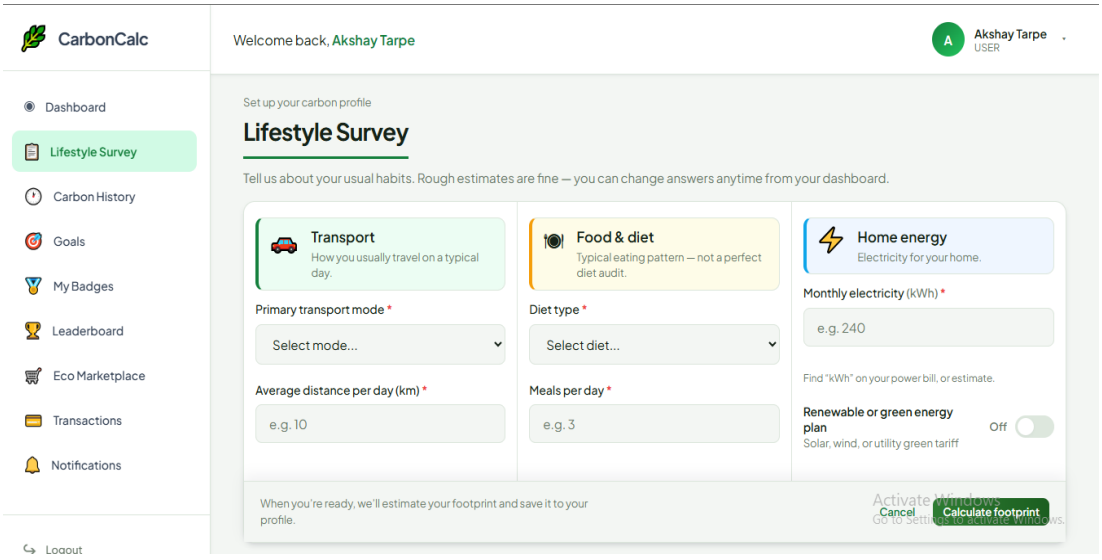


Fig. 5.5: Lifestyle Survey Page

5.1.5 Carbon History Module

The Carbon History Module maintains historical carbon emission records submitted by users. Previous emission data is displayed using tables, charts, and graphical trend

analysis for better sustainability monitoring. The module helps users analyze their past environmental activities and identify patterns related to carbon generation over different time periods. By visualizing historical sustainability data, users can evaluate improvement progress and make more informed decisions toward reducing future carbon emissions.

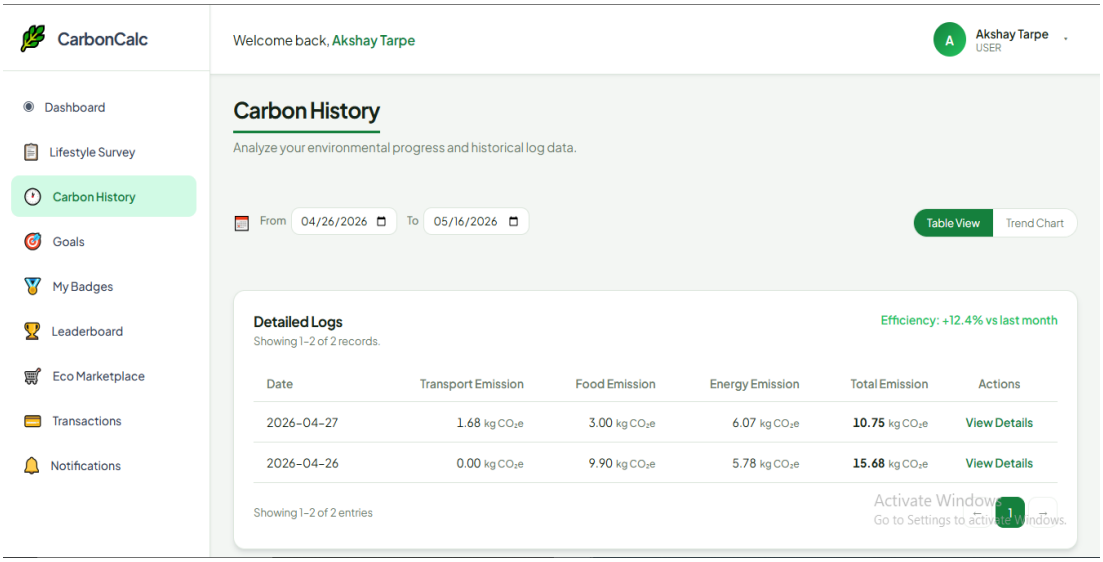


Fig. 5.6: Carbon History Page

Users can compare environmental performance across different time periods and identify areas where lifestyle improvements are required. Historical tracking improves long-term environmental awareness and helps users monitor sustainability progress consistently. The module retrieves stored data from PostgreSQL databases using backend APIs and displays it through dynamic dashboard visualizations.

5.1.6 Goal Tracking Module

5.1.6.1 User Goal Tracking

The Goal Tracking Module allows users to create personal sustainability goals such as reducing electricity usage, minimizing fuel consumption, or lowering carbon emissions. Users can define target values and continuously monitor progress through dashboard indicators and graphical summaries. The sustainability goal creation interface developed for creating and target tracking goals is shown in Fig. 5.7.

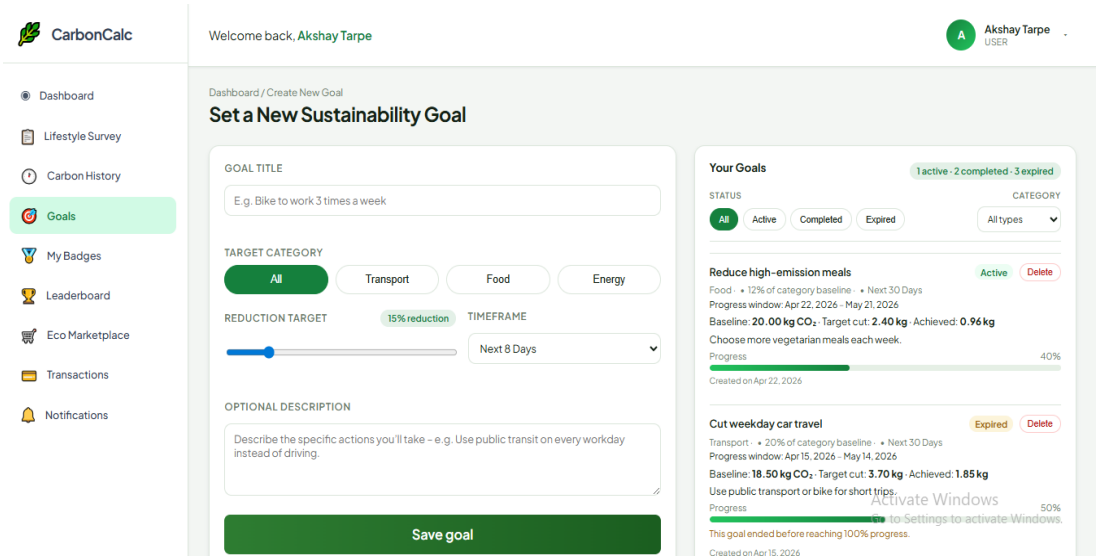


Fig. 5.7: Goal Creation Page

Goal tracking improves user motivation and encourages environmentally responsible behavior through measurable sustainability objectives.

5.1.6.2 Admin Goal Monitoring

The administrator interface allows monitoring of user sustainability goals and completion statistics. Administrators can review user progress, verify achievement completion, and analyze overall system engagement related to environmental tracking activities. The sustainability goal management interface developed for progress monitoring and target tracking is shown in Fig. 5.8.

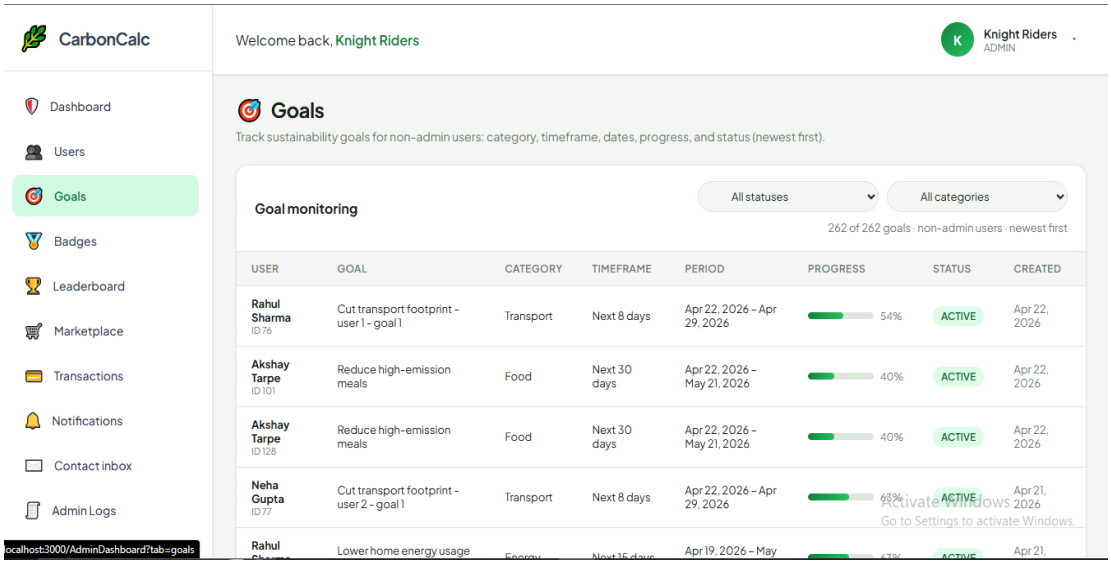


Fig. 5.8: Goal Monitoring page

This module improves administrative monitoring and helps maintain organized sustainability management throughout the platform.

5.1.7 Badge and Gamification Module

5.1.7.1 User Badges

The User Badge Module rewards users for achieving sustainability milestones and maintaining environmentally friendly habits. Users receive achievement badges after completing goals, reducing emissions, or actively participating within the platform. These badges motivate users to remain engaged with sustainability tracking activities and encourage continuous participation in environmental awareness programs. The module also improves user interaction by transforming carbon monitoring into an interactive and achievement-oriented experience. The achievement badge and reward system implemented for improving user engagement is illustrated in Fig. 5.9.

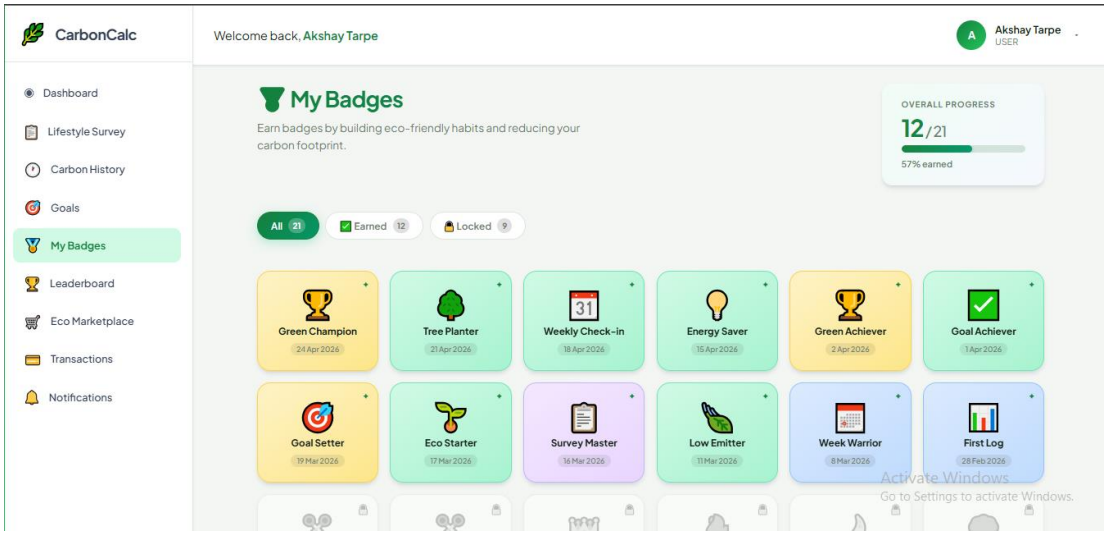


Fig. 5.9: User Badges Page

Gamification features improve user engagement and transform sustainability monitoring into an interactive experience.

5.1.7.2 Admin Badge Management

Administrators can create, manage, and assign badges through the Badge Management Module. The admin panel allows modification of badge names, descriptions, reward criteria, and achievement conditions. These functionalities help maintain organized achievement tracking and improve user motivation through structured sustainability rewards. The module also enables administrators to monitor badge distribution and manage engagement activities efficiently within the platform. The achievement badge creation form is illustrated in Fig. 5.10.

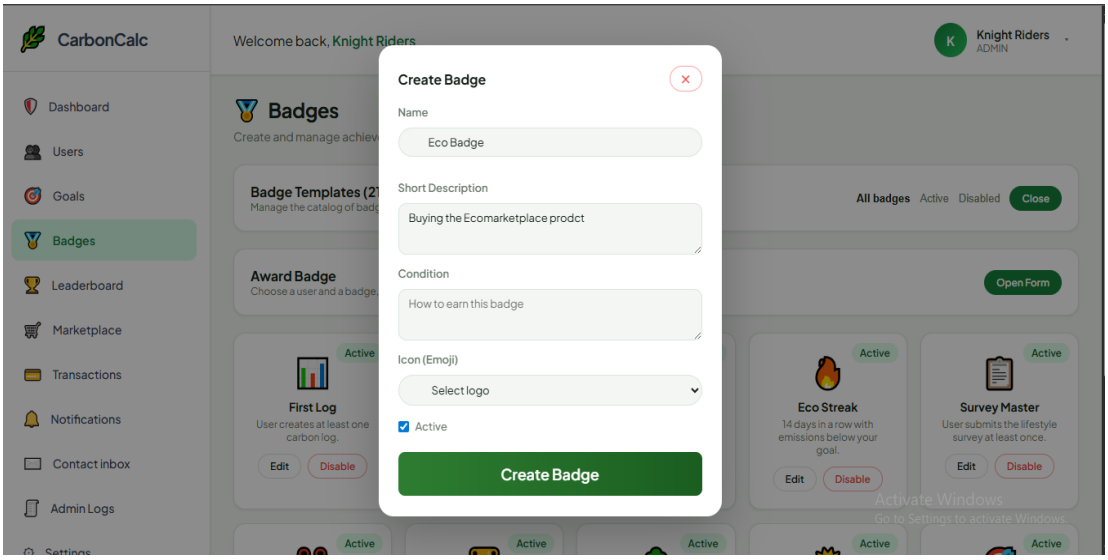


Fig. 5.10: Admin Badges

This functionality improves customization and supports motivational sustainability campaigns within the application.

5.1.8 Leaderboard Module

The Leaderboard Module ranks users according to sustainability performance and carbon reduction achievements. Rankings are generated based on environmental activities, completed goals, and overall sustainability scores. Leaderboard systems encourage healthy competition among users and improve continuous participation within the CarbonCalc platform. Dynamic ranking calculations are processed through backend APIs and displayed using responsive frontend tables and dashboard components. The leaderboard interface designed for comparing sustainability performance rankings among users is shown in Fig. 5.11.

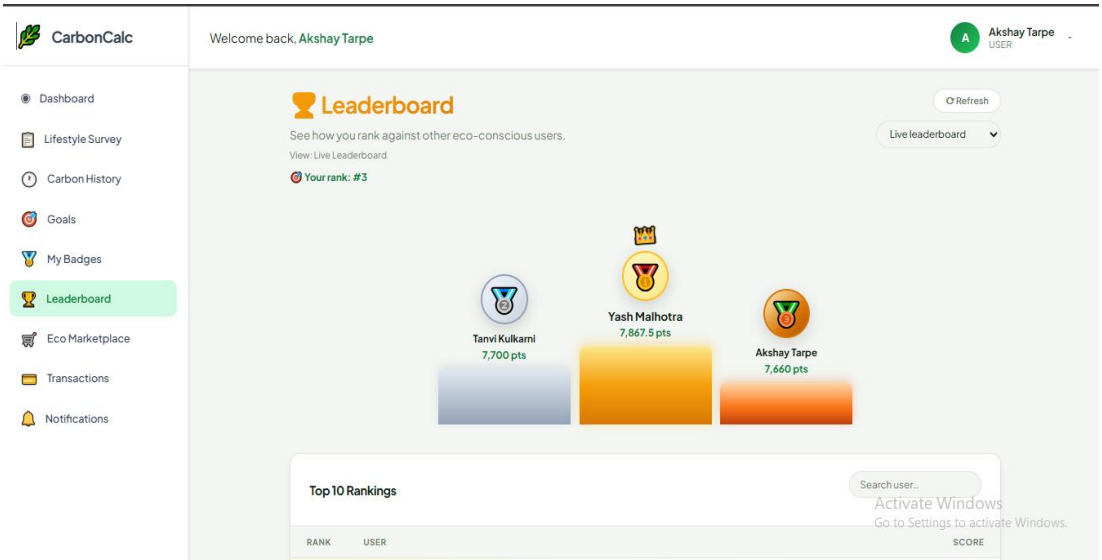


Fig. 5.11: Leaderboard Page

5.1.9 Eco Marketplace Module

5.1.9.1 User Marketplace

The Eco Marketplace allows users to browse and purchase eco-friendly products, sustainability initiatives, and carbon offset services. Marketplace features encourage users to participate in environmental improvement activities beyond monitoring alone. The module promotes awareness regarding sustainable products and environmentally responsible consumption practices through interactive digital features. Users can explore different categories of eco-friendly items and sustainability programs that support carbon reduction activities and green living initiatives. The Eco Marketplace interface designed for promoting eco-friendly products and sustainability awareness is shown in Fig. 5.12.

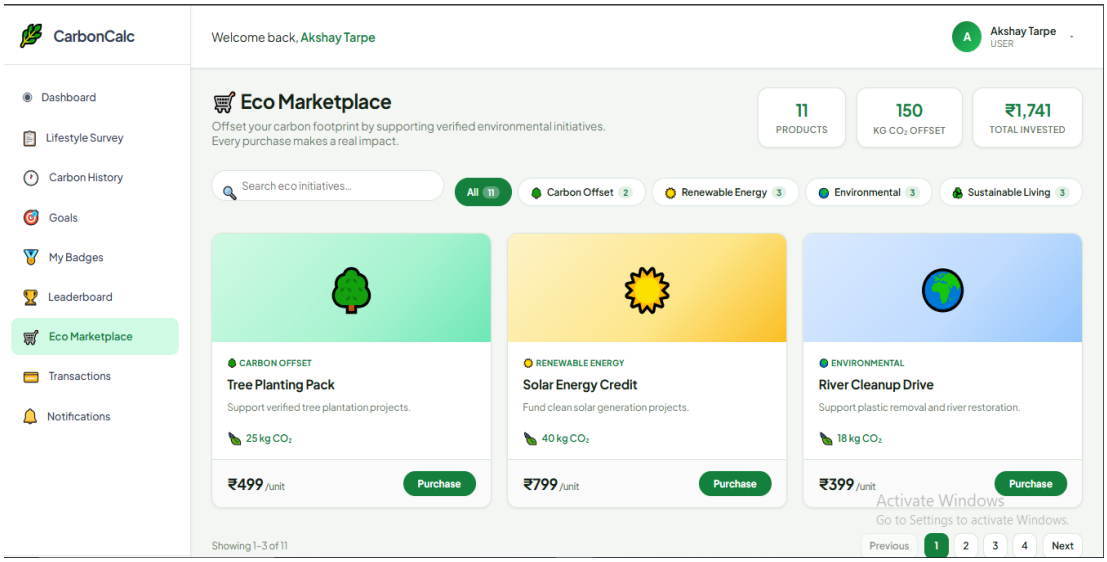


Fig. 5.12: User Eco Marketplace Page

Product listings, descriptions, and purchase options are displayed through organized frontend layouts integrated with backend marketplace APIs.

5.1.9.2 Admin Marketplace Management

Administrators can create and manage marketplace products through the admin marketplace module. Product details such as pricing, descriptions, environmental categories, and availability status can be controlled through this interface. The module helps maintain organized product management and supports efficient monitoring of sustainability-related marketplace activities within the platform. Administrators can also update product information, manage inventory visibility, and promote environmentally friendly initiatives through centralized marketplace controls. The Eco

Marketplace management page designed for creating eco-friendly products and sustainability awareness is shown in Fig. 5.13.

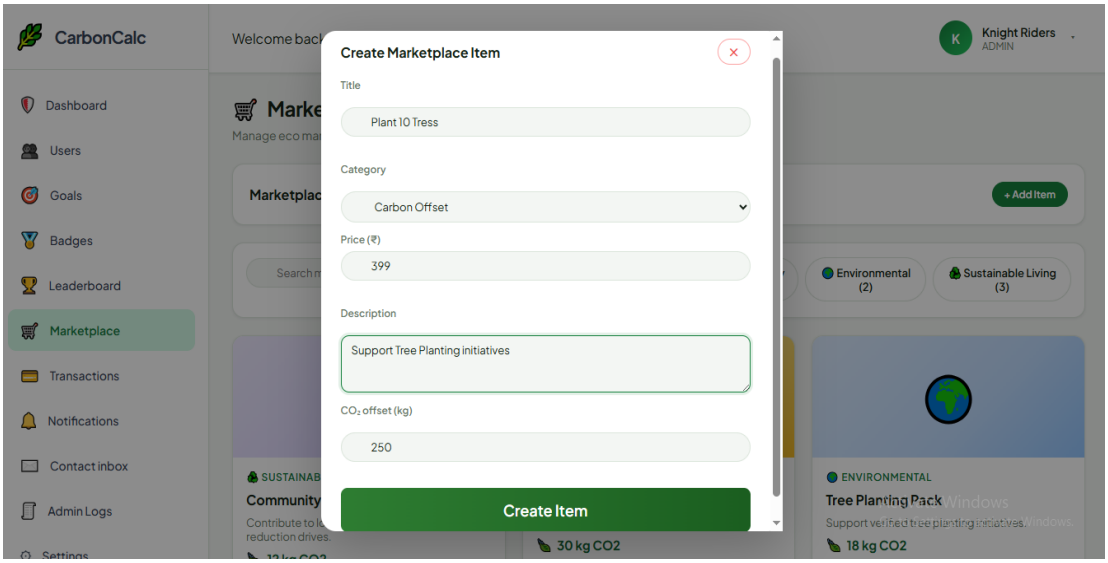


Fig. 5.13: Eco-Marketplace Management Page

The module helps maintain marketplace operations and supports environmental sustainability initiatives.

5.1.10 Transaction Management Module

The Transaction Management Module stores purchase records, payment details, and carbon offset transaction history generated through marketplace activities. Users can monitor completed transactions and purchase history, while administrators can track payment records and marketplace performance. Secure database handling and transaction validation improve reliability and maintain accurate financial records within the system. The Transaction Page is shown in Fig. 5.14.

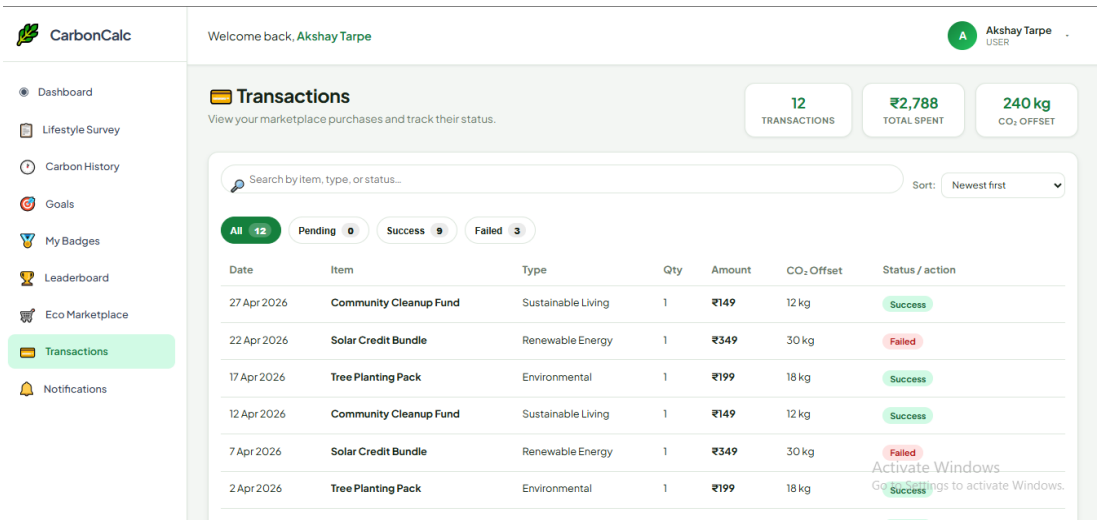


Fig. 5.14: Transaction Page

5.1.11 Notification Module

5.1.11.1 User Notifications

The Notification Module provides alerts and updates related to sustainability goals, achievement badges, leaderboard rankings, and environmental activities. The module helps users stay informed about recent platform activities, sustainability progress, and important system updates through real-time notifications. Notification management improves user engagement by encouraging continuous participation and timely interaction with sustainability tracking functionalities. The notification management interface developed for displaying user activity updates and alerts is illustrated in Fig. 5.15.

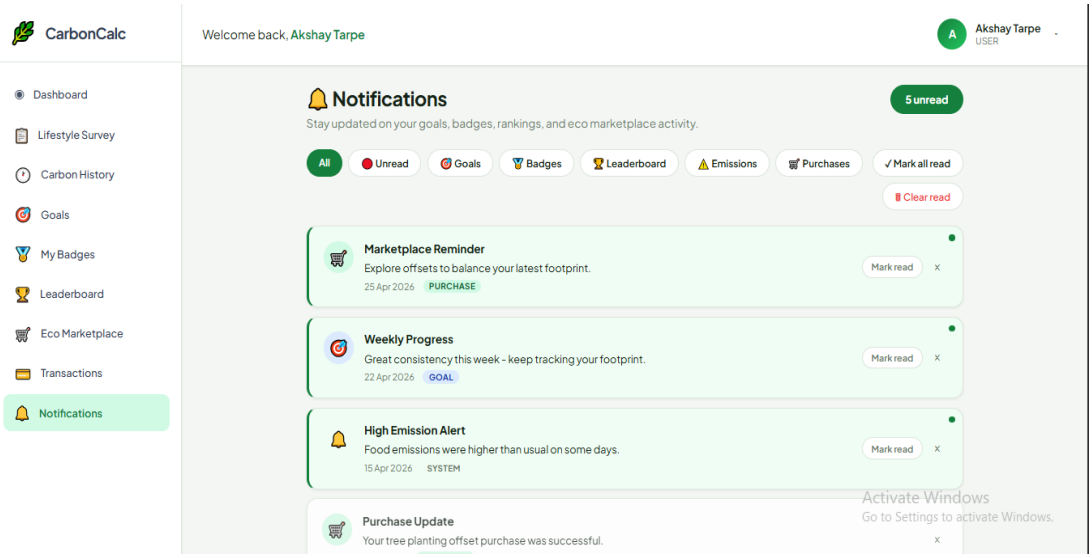


Fig. 5.15: User Notification Page

Notifications improve user engagement and help users remain informed about important platform activities and sustainability progress.

5.1.11.2 Admin Notification Creation

Administrators can create and distribute notifications related to environmental awareness campaigns, goal reminders, system updates, and platform announcements. The module helps administrators communicate important sustainability information efficiently to all users within the platform. Notification management also improves user engagement by providing timely updates regarding environmental activities, achievement events, and application-related announcements. The notification management interface developed for displaying user activity updates and alerts is illustrated in Fig. 5.16.

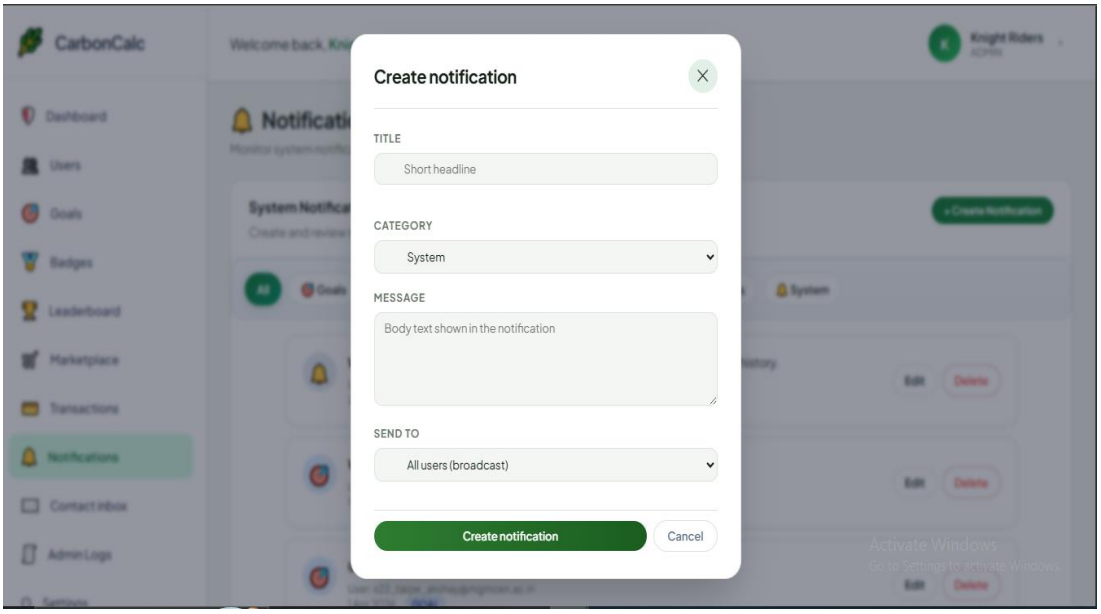


Fig. 5.16: Notification Management Page

5.1.12 User Management Module

The User Management Module allows administrators to monitor registered users, manage account status, and perform moderation activities. The user profile management interface implemented for account handling and personalization is illustrated in Fig. 5.17.

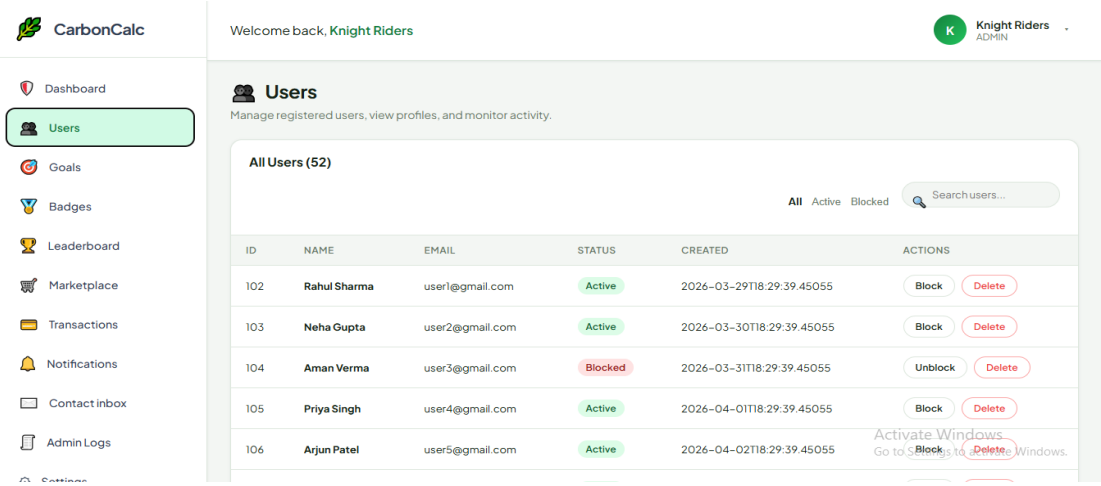


Fig. 5.17: User Management Page

Administrative functionalities include viewing user records, activating or disabling accounts, and monitoring platform participation statistics. This module improves overall system administration and user control management. Administrators can efficiently supervise user activities, manage account-related operations, and maintain secure platform accessibility through centralized administrative controls. The

module also supports monitoring of user engagement, sustainability participation trends, and overall application performance for better system management.

5.1.13 Admin Logs Module

The Admin Logs Module records administrative activities performed within the platform for security and monitoring purposes. The module maintains detailed records related to user management operations, notification activities, badge creation, marketplace updates, and other administrative actions performed within the system. These logs help improve system transparency, security monitoring, and accountability by tracking important platform activities systematically. The Admin Logs interface used for monitoring administrative activities is shown in Fig. 5.18.

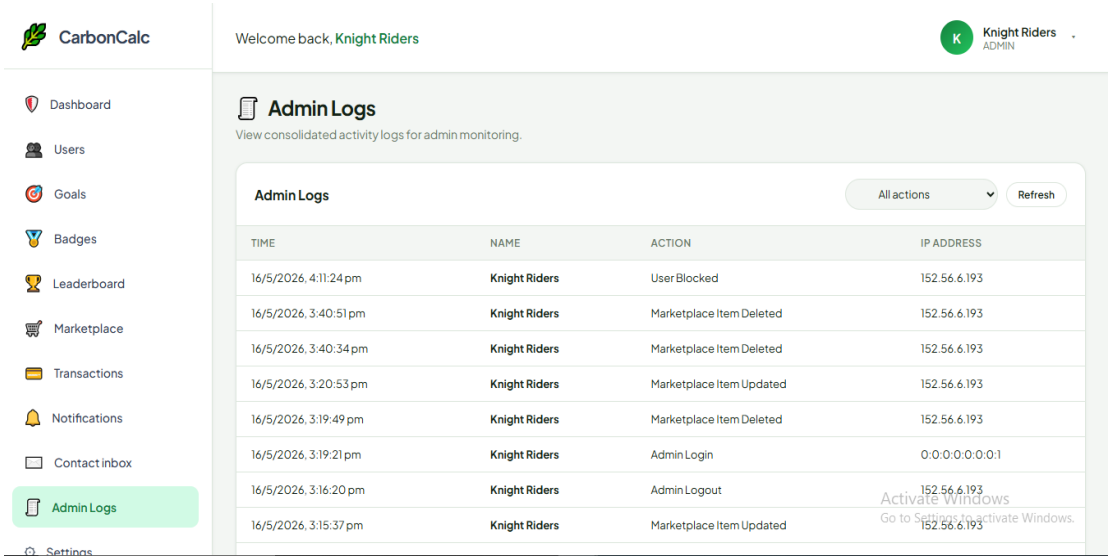


Fig. 5.18: Admin Logs Page

Logs help track actions such as user management updates, badge creation, notification handling, and marketplace operations. This improves accountability and system transparency.

5.1.14 Settings and Configuration Module

The Settings and Configuration Module allows administrators to manage system-level configurations related to carbon calculation factors, platform preferences, and application maintenance settings. Administrative configuration improves flexibility and supports future scalability and environmental calculation customization within the system. The admin settings interface developed for managing application configurations and administrative functionalities is shown in Fig. 5.19.

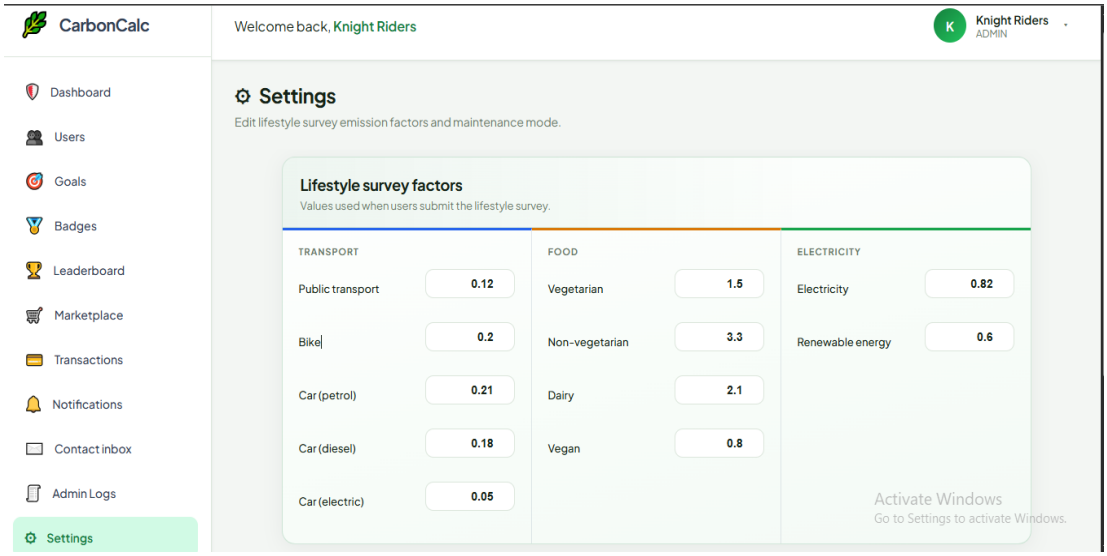


Fig. 5.19: Admin Settings Page

5.2 Limitations of the System

Despite successful implementation and stable system performance, the Personal Carbon Footprint Application has certain limitations related to data collection, environmental estimation accuracy, advanced feature availability, and scalability constraints. Identifying these limitations is important because it helps define areas for future improvement and system enhancement. One of the major limitations of the application is its dependency on manual user inputs. Users are required to enter transportation details, electricity usage, food habits, and sustainability-related activities manually through frontend survey interfaces. Since automatic environmental data collection mechanisms were not implemented, user-provided data may vary in accuracy and consistency.

Another limitation involves carbon emission estimation accuracy. The application uses simplified environmental emission factors and approximate calculation methods to estimate carbon footprint values. Real-world environmental analysis depends on multiple dynamic variables and scientific measurement techniques, which were beyond the scope of the current implementation. Therefore, the system should be considered primarily as an environmental awareness and sustainability monitoring platform rather than a scientifically precise carbon measurement system.

Advanced functionalities such as AI-based recommendations, IoT-integrated automatic tracking, smart energy monitoring, real-time environmental analytics, multilingual support, and organization-level sustainability management were not fully implemented due to project timeline and resource limitations. These advanced features

require additional infrastructure, external integrations, and extended development effort. The application also depends on internet connectivity because communication between frontend interfaces, backend APIs, and PostgreSQL database systems occurs through web-based architecture. Without stable internet access, users cannot interact with dashboard analytics, authentication systems, or sustainability monitoring functionalities effectively.

Scalability under large-scale real-world deployment conditions was not tested extensively during implementation. Although the modular architecture supports future scalability, additional optimization and infrastructure management would be required for handling large numbers of concurrent users and real-time environmental data processing.

5.3 Future Scope

Although the Personal Carbon Footprint Application successfully fulfills its current objectives, several advanced features and improvements can be implemented in the future to enhance system functionality, automation, and user experience.

One of the most important future improvements is the integration of Artificial Intelligence (AI). AI algorithms can analyze user behavior and provide personalized sustainability recommendations. For example, the system could suggest environmentally friendly transportation methods, electricity-saving techniques, or optimized sustainability goals based on user activity patterns. Machine learning models could also predict future carbon emission trends and provide intelligent environmental insights.

Another major enhancement is IoT (Internet of Things) integration. Smart devices and sensors could automatically collect environmental data such as electricity usage, fuel consumption, or appliance activity. This would reduce dependency on manual user inputs and improve carbon calculation accuracy significantly. Real-time monitoring through IoT integration would make the application more practical for daily use.

Mobile application development is another important future scope area. Currently, the system operates as a web-based platform, but dedicated Android and iOS applications could improve accessibility and user convenience. Mobile notifications and reminders could also increase user engagement and sustainability tracking consistency.

Advanced analytics and reporting features can also be added in future versions. More detailed graphical reports, monthly sustainability comparisons, and AI-generated environmental summaries could improve user understanding further. Data visualization techniques such as predictive graphs and heat maps could make analytics more interactive and informative.

Multilingual support is another important enhancement for improving accessibility. Currently, the application uses English interfaces, but regional language support could make the platform accessible to wider audiences and improve usability for users from different regions.

5.4 Internship Experience

During the internship period, practical exposure to modern full-stack web development technologies was gained. The internship helped improve technical knowledge related to React.js, Spring Boot, PostgreSQL, REST APIs, authentication handling, responsive frontend development, and database integration. Team collaboration, debugging, API testing, and problem-solving skills were also improved through real-time project implementation activities.

The internship also improved communication skills, teamwork coordination, project planning, and real-time problem-solving abilities during collaborative development activities.

CONCLUSION

The Personal Carbon Footprint Application was successfully developed as an interactive sustainability monitoring platform to help users track, analyze, and manage their carbon emissions through digital analytics and environmental awareness features. The system enables users to monitor activities related to transportation, electricity consumption, and lifestyle habits while providing graphical dashboards, sustainability reports, and carbon emission analytics. The application was designed with the objective of encouraging environmentally responsible behavior and improving awareness regarding climate change and pollution caused by daily human activities.

During the internship period, practical implementation work was performed for developing frontend interfaces, backend APIs, authentication systems, dashboard analytics, database integration, and sustainability monitoring functionalities. Important full-stack web development concepts such as REST API communication, responsive frontend design, backend processing, secure authentication handling, database management, debugging, and modular application development were implemented successfully throughout the project lifecycle. The internship provided valuable practical exposure to real-time software development environments and improved technical knowledge related to modern web application technologies.

The project also helped improve professional skills such as teamwork coordination, communication, project planning, debugging, API testing, and problem-solving through collaborative development activities. Working on a sustainability-focused application provided a better understanding of how software systems can contribute toward solving environmental and social challenges through digital solutions.

REFERENCES

[1] React.js Documentation, "React – The Library for Web and Native User Interfaces", Available: https://react.dev

[2] Spring Boot Documentation, "Spring Boot Project Documentation", Available: https://spring.io/projects/spring-boot

[3] PostgreSQL Global Development Group, "PostgreSQL Documentation", Available: https://www.postgresql.org/docs/

[4] Postman Inc., "Postman API Platform Documentation", Available: https://learning.postman.com/docs/introduction/overview/

[5] Apache Software Foundation, "Maven Project Documentation", Available: https://maven.apache.org/guides/

[6] PlantUML Documentation, "PlantUML – Open Source UML Tool", Available: https://plantuml.com

[7] I. Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016, ISBN: 978-0133943030.

[8] R. S. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw Hill Education, 2014, ISBN: 978-0078022128.

[9] Ethan Brown, *Learning JavaScript: JavaScript Essentials for Modern Application Development*, 3rd Edition, O'Reilly Media, 2016, ISBN: 978-1491914915.

[10] Environmental Protection Agency, "Carbon Footprint and Sustainability Resources", Available: https://www.epa.gov/carbon-footprint-calculator

[11] JWT Introduction, "JSON Web Token Documentation", Available: https://jwt.io/introduction

[12] GitHub Documentation, "GitHub Platform Documentation", Available: https://docs.github.com/

DrillBit

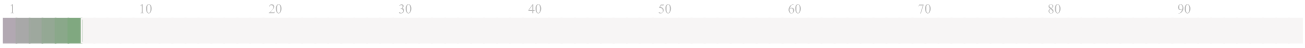
The Report is Generated by DrillBit Plagiarism Detection Software

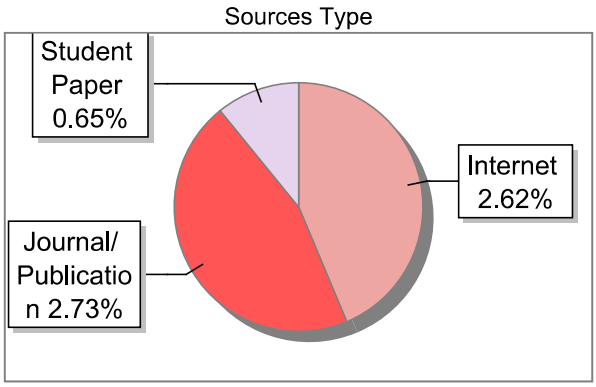
Submission Information

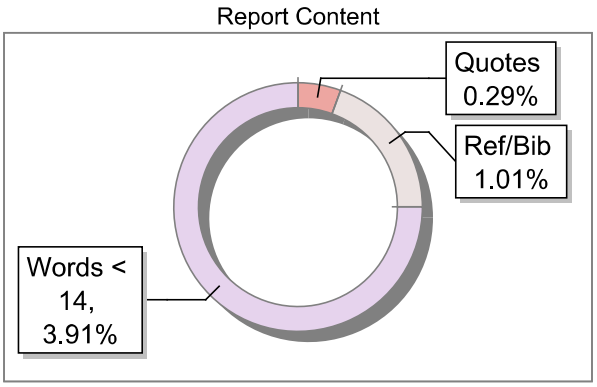
Author Name	Tarpe Akshay Shankar
Title	akshay tarpe b-tech cse(A)
Paper/Submission ID	5764580
Submitted by	rajurkar_am@mgmcen.ac.in
Submission Date	2026-05-30
Total Pages, Total Words	64, 14151
Document type	Project Report

Result Information

Similarity **6 %**







Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

DrillBit

DrillBit Similarity Report

6	69	A
SIMILARITY %	MATCHED SOURCES	GRADE

A-Satisfactory (0-10%)
B-Upgrade (11-40%)
C-Poor (41-60%)
D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	REPOSITORY - Submitted to Exam section VTU on 2026-02-12 16-12 4402458	<1	Student Paper
2	ec.europa.eu	<1	Internet Data
3	moam.info	<1	Internet Data
4	REPOSITORY - Submitted to Exam section VTU on 2026-02-12 16-12 4402459	<1	Student Paper
5	dochero.tips	<1	Internet Data
6	ijsrst.com	<1	Publication
7	link.springer.com	<1	Internet Data
8	www.adaptation-fund.org	<1	Publication
9	jecrcu.academia.edu	<1	Internet Data
10	An efficiency-based allocation of carbon emissions allowance A case s by Du-2020	<1	Publication
11	optimizeias.com	<1	Publication
12	waseda.repo.nii.ac.jp	<1	Publication

13	Markets of a single customer exploiting conceptual developments in market segme by Kara-1997	<1	Publication
14	moam.info	<1	Internet Data
15	Student Archives Data	<1	Publication
16	www.bgc-jena.mpg.de	<1	Internet Data
17	www.linkedin.com	<1	Internet Data
18	dx.doi.org	<1	Internet Data
19	moam.info	<1	Internet Data
20	springeropen.com	<1	Internet Data
21	download.data.public.lu	<1	Internet Data
22	ebooks.toucantoco.com	<1	Internet Data
23	Surface-initiated phase transition in solid hydrogen under the high-p, by Lei, Haile; Lin, We - 2018	<1	Publication
24	www.dx.doi.org	<1	Publication
25	1library.co	<1	Internet Data
26	al-kindipublisher.com	<1	Publication
27	ijcrt.org	<1	Publication
28	besjournals.onlinelibrary.wiley.com	<1	Internet Data
29	docplayer.net	<1	Internet Data
30	e-journal.trisakti.ac.id	<1	Internet Data

31	Financial Sector Reforms in Indonesia and South Korea in 1980s and Ear by Akimov-2010	<1	Publication
32	Gender Differences in Space-Time Constraints by Mei-P-2000	<1	Publication
33	Improving usability of password authentication using keystroke dynamics by Walte-2018	<1	Publication
34	infonomics-society.org	<1	Publication
35	repository.unsoed.ac.id	<1	Publication
36	rsisinternational.org	<1	Internet Data
37	Thesis Submitted to Shodhganga Repository	<1	Publication
38	www.deel.com	<1	Internet Data
39	xn----8sbeaobxmdwlbbckrtgii.xn--p1ai	<1	Internet Data
40	academicworks.cuny.edu	<1	Publication
41	A comparative study of student perceptions on generative AI in programming educ, by Oyelere, Solomon Sunday, Yr-2025	<1	Publication
42	A high-accuracy, real-time, intelligent material perception system with a machine-learning-motivate, by Wei, Xiao, Yr-2022	<1	Publication
43	clutejournals.com	<1	Publication
44	coek.info	<1	Internet Data
45	Developing a Software Module to Automate the Process of Categorizing Objects of By Julija Goncharenko, Andrej Go, Yr-2019,10	<1	Publication
46	digital.lib.washington.edu	<1	Publication
47	docshare.tips	<1	Internet Data

48	Do brands make consumers happy- A masstige theory perspective by Kumar-2021	<1	Publication
49	ePOCT+ and the medAL-suite Development of an electronic clinical decision supp By Rainer Tan, Ludovico Cobuccio, Yr-2022,9,4	<1	Publication
50	geeksforgeeks.org	<1	Internet Data
51	georgian.edu	<1	Publication
52	Improvements in adverse drug reaction prediction By Wanyu Zhu, Yusen Wang, Yuzhou, Yr-2023,12	<1	Publication
53	kth.diva-portal.org	<1	Publication
54	library.oapen.org	<1	Publication
55	mdpi.com	<1	Internet Data
56	moam.info	<1	Internet Data
57	Modeling and forecasting CO2 emissions in China and its regions using a novel ARIMA-LSTM model, by Wen, Tingxin, Yr-2023	<1	Publication
58	nsc.anu.edu.au	<1	Publication
59	nu.edu	<1	Internet Data
60	pmc.ncbi.nlm.nih.gov	<1	Internet Data
61	saranathan.ac.in	<1	Publication
62	Study on numerical method to predict wheelrail profile evolution due to wear by Chan-2010	<1	Publication
63	Submitted to National Law Institute University Bhopal on 2026-03-22 20-09 5375288	<1	Student Paper
64	www.astrobio.net	<1	Internet Data

65	www.global.toshiba	<1	Publication
66	www.itu.int	<1	Publication
67	www.mdpi.com	<1	Internet Data
68	www.nso-journal.org	<1	Publication
69	www.sciencedirect.com	<1	Internet Data